

MACHINE LEARNING TUTORIAL

从原理到实践 · 8章 + 3附录完整手册

机器学习 全面教程

覆盖线性回归、决策树、聚类、集成学习
SVM、神经网络、降维与特征工程

本教程系统讲解机器学习核心算法的原理、流程与实现。每章含计算示例、工程图表与应用实例（房价预测、贷款审批、客户分群、图像压缩等），配套12个Jupyter Notebook与3个数学/框架附录。

8章+3附录

工程图表

12 Notebook

计算示例

应用实例

VERSION v1.0 · 2026.08

AUDIENCE ML 工程师 · 数据科学家 · 学生

CONTENT PDF教程 + Notebook代码包

目录

第一章 机器学习概览	5
1.1 什么是机器学习	5
1.2 机器学习的三大类型	5
1.3 机器学习的工作流程	6
1.4 与后续章节的关联	7
1.5 现代发展趋势	7
第二章 线性回归	7
2.1 算法原理：最小二乘法与 MSE	7
2.2 梯度下降法	8
2.3 正则化：Lasso 与 Ridge	9
2.4 算法实现描述	9
2.5 计算示例：三个样本的手算	10
2.6 应用实例：加州房价预测	11
2.7 代码实现：sklearn LinearRegression	11
2.8 现代发展	12
第三章 决策树	12
3.1 算法原理：三大经典算法	12
3.2 决策树结构	14
3.3 剪枝：预剪枝 vs 后剪枝	14
3.4 算法实现描述	14
3.5 计算示例：用信息增益选分裂特征	15
3.6 应用实例：贷款审批决策树	16
3.7 代码实现：sklearn DecisionTreeClassifier	16
3.8 现代发展	17
第四章 聚类分析	18
4.1 K-Means 原理	18

4.2 DBSCAN 原理	19
4.3 层次聚类	19
4.4 评估指标	20
4.5 算法实现描述	20
4.6 计算示例: 5 个点的 K-Means 手算	21
4.7 应用实例 1: 客户 RFM 分群	21
4.8 应用实例 2: 图像颜色量化压缩	22
4.9 应用实例 3: 异常检测	22
4.10 代码实现: sklearn KMeans 与 DBSCAN	23
4.11 现代发展	23
第五章 集成学习	25
5.1 三大集成范式	25
5.2 Bagging 与随机森林	25
5.3 Boosting 与梯度提升树	26
5.4 算法实现描述	26
5.5 计算示例: 简单 Bagging 投票手算	27
5.6 应用实例: 特征重要性分析	27
5.7 代码实现: sklearn 与 XGBoost	28
5.8 现代发展与小结	29
第六章 支持向量机	30
6.1 算法原理: 最大间隔与支持向量	30
6.2 软间隔与正则化	30
6.3 核函数: 从线性到非线性	31
6.4 对偶问题 (简述)	31
6.5 算法实现描述	31
6.6 计算示例: 2D 简单 SVM 间隔计算	32
6.7 应用实例: 手写数字识别	33
6.8 代码实现: sklearn SVC	33
6.9 现代发展与小结	33

第七章 神经网络基础	34
7.1 神经元模型：感知机与多层感知机	34
7.2 激活函数	34
7.3 前向传播与反向传播	35
7.4 损失函数	35
7.5 优化器	35
7.6 算法实现描述	35
7.7 计算示例：2 层网络前向传播手算	37
7.8 应用实例：MNIST 手写识别	37
7.9 代码实现：PyTorch MLP	37
7.10 现代发展与小结	38
第八章 降维与特征工程	39
8.1 PCA 主成分分析	39
8.2 t-SNE 与 UMAP	39
8.3 特征工程	40
8.4 算法实现描述	40
8.5 计算示例：2D $\rightarrow$ 1D PCA 投影手算	41
8.6 应用实例：高维数据可视化	41
8.7 代码实现：sklearn PCA 与 t-SNE	42
8.8 现代发展与小结	42
附录 A 数学基础	43
A.1 线性代数	43
A.2 概率论	43
A.3 微积分	44
A.4 信息论	44
附录 B 集成学习框架	46
B.1 Scikit-learn	46
B.2 XGBoost	46
B.3 LightGBM	46

B.4 CatBoost	46
B.5 框架对比	47
附录 C PyTorch 与 Transformers	48
C.1 PyTorch 基础	48
C.2 数据加载: Dataset 与 DataLoader	48
C.3 HuggingFace Transformers	48
C.4 代码示例: BERT 文本分类微调	49
C.5 进阶话题与小结	49

第一章 机器学习概览

在数据爆炸的时代，全球每天产生的数据量早已超过 EB 量级，从电商点击流、物联网传感器、社交网络互动到基因组测序，数据的增长速度远超人工分析的能力边界。传统编程范式是“人写规则、机器执行”，但当规则过于复杂或根本无法显式描述时——例如识别人脸、预测用户兴趣、判断一封邮件是否为垃圾邮件——这种范式就走到了尽头。机器学习（Machine Learning, ML）正是为应对这一痛点而生：让算法从数据中自动归纳模式，再用学到的模式对新数据做出预测或决策。本章将建立整体认知框架，为后续章节中线性回归、决策树、聚类具体算法的学习打下基础。

1.1 什么是机器学习

Tom Mitchell 给出的经典定义是：“如果一个程序在任务 T 上的性能 P 随着经验 E 的增加而提升，就称该程序从经验 E 中学习。”**经验**对应历史数据，**任务**是预测、分类或聚类等，**性能**则用准确率、误差、轮廓系数等指标度量。与传统统计建模相比，机器学习更强调预测精度与可扩展性，弱化模型的可解释性与分布假设。与规则引擎相比，机器学习无需显式编码业务规则，而是从样本中自动提取特征-目标映射关系。

机器学习之所以在近十年取得突破，依赖三大要素的合力：一是**数据**——互联网与移动设备让标注数据成本大幅下降；二是**算力**——GPU/TPU 让大规模矩阵运算成为日常；三是**算法**——从逻辑回归到 Transformer，模型表达能力持续提升。三者中任一缺失，机器学习都无法落地。

1.2 机器学习的三大类型

依据训练数据是否带标签以及反馈形式，机器学习通常分为三大类：监督学习、无监督学习与强化学习。这种分类并非互斥——半监督学习、自监督学习正在模糊三者边界，但作为入门框架，三大类型仍然是最实用的认知坐标系。

- **监督学习 (Supervised Learning)**：训练数据为 (x, y) 对，目标是从输入 x 预测标签 y 。代表任务：分类（垃圾邮件识别、图像分类）、回归（房价预测、销量预测）。代表算法：线性回归、逻辑回归、决策树、SVM、神经网络。
- **无监督学习 (Unsupervised Learning)**：训练数据只有 x 没有 y ，目标是发现数据内在结构。代表任务：聚类（客户分群）、降维（PCA）、密度估计、关联规则。代表算法：K-Means、DBSCAN、层次聚类、Apriori。
- **强化学习 (Reinforcement Learning)**：智能体通过与环境交互、依据奖励信号学习最优策略。代表任务：游戏 AI（AlphaGo）、机器人控制、推荐系统、RLHF。代表算法：Q-Learning、DQN、PPO、SAC。



图 1.1 机器学习三大类型对比：监督学习有标签，无监督学习无标签，强化学习靠奖励

1.3 机器学习的工作流程

一个完整的机器学习项目并非只是"训练一个模型"，而是包含从业务理解到线上部署的完整流水线。业界通行的流程大致可分为六步，每一步的质量都直接决定最终效果：

- **1. 业务理解与数据收集**：明确要解决的问题、可用数据源、成功指标。常见数据源包括数据库、API、日志、爬虫、人工标注。
- **2. 数据清洗**：处理缺失值、异常值、重复样本、类型不一致。脏数据是模型表现差的首要原因，"Garbage in, garbage out"是 ML 的铁律。
- **3. 特征工程**：从原始数据构造模型可用特征，包括数值归一化、类别编码、时间特征提取、交叉特征、特征选择。特征工程的天花板往往决定模型的天花板。
- **4. 模型训练**：选择算法、划分训练/验证/测试集、调参、交叉验证。注意防止数据泄漏（Data Leakage）。
- **5. 模型评估**：分类看准确率/精确率/召回率/F1/AUC；回归看 MSE/MAE/R²；聚类看轮廓系数/DB 指数。必须在测试集上评估，不可用训练集指标。
- **6. 部署与监控**：将模型上线为 API 或流式服务，监控预测分布漂移（Data Drift）、特征漂移、性能衰减，建立再训练机制。

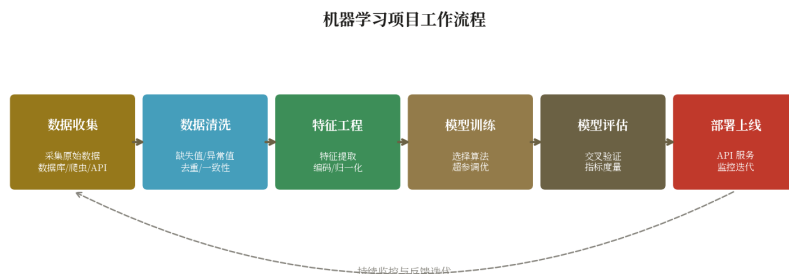


图 1.2 机器学习项目标准工作流程：从业务理解到部署监控的六步闭环

[技巧] 在工业实践中，数据清洗与特征工程通常占项目总时长的 60%-80%，模型训练只占 10%-20%。初学者常沉溺于调参而忽视数据质量，这是新手最常见的认知误区。优秀工程师的口号是"先看数据，再看模型"。

1.4 与后续章节的关联

本教程按照"由浅入深、由监督到无监督、由模型到工程"的脉络组织。第二章以**线性回归**开篇，它是最简单也最具启发性的监督学习算法，从中可推导出损失函数、梯度下降、正则化等贯穿全书的核心概念。第三章讨论**决策树**，从基于规则的可解释模型出发，引出信息增益、剪枝、以及后续集成学习（随机森林、GBDT、XGBoost）的基础。第四章进入**聚类分析**，是无监督学习的代表，从 K-Means 到 DBSCAN，展示如何在没有标签的数据中发现结构。后续章节将依次展开 SVM、神经网络、深度学习、表示学习、强化学习等主题，形成完整的机器学习知识图谱。

1.5 现代发展趋势

机器学习的边界正在快速扩张，几个值得关注的现代趋势包括：

- **AutoML**：自动化特征工程、模型选择、超参调优。代表工具：Auto-sklearn、TPOT、H2O AutoML、Google Cloud AutoML。让非专家也能用上 ML。
- **MLOps**：将 DevOps 思想引入 ML 生命周期，解决模型版本管理、流水线编排、监控漂移、灰度发布等问题。代表工具：MLflow、Kubeflow、Metaflow、Weights & Biases。
- **大模型时代**：以 GPT、Claude、Gemini 为代表的大语言模型正在重塑 ML 范式。预训练+微调（含 PEFT/LoRA）+ RLHF/RLAIF 已成为新范式，"模型即服务"成为常态。
- **自监督学习**：利用数据自身结构作为监督信号（如掩码语言建模、对比学习），大幅降低对人工标注的依赖，是 BERT、ViT、CLIP 成功的关键。
- **负责任 AI**：公平性、可解释性、隐私保护、稳健性成为工业级 ML 系统的硬性指标。相关法规（如 EU AI Act）正在落地。

尽管技术快速演进，但本章介绍的工作流程与三大类型分类依然是理解所有现代方法的基础。无论 GPT 多复杂，其训练仍包含数据、特征（Token）、模型、评估、部署的全流程；无论 Transformer 多强大，它依然在解决监督学习或自监督学习框架下的问题。掌握基础原理，是应对技术潮流更迭的锚点。

第二章 线性回归

房价预测是机器学习最经典的应用场景之一：给定房屋面积、卧室数量、房龄、地段等特征，能否预测其合理售价？这是一个典型的**回归问题**——目标变量是连续值（价格），特征是多维实数。线性回归（Linear Regression）是解决这类问题最基础也最有效的算法：它假设特征与目标之间存在近似线性的关系，通过最小化预测误差确定最优参数。尽管真实世界中关系很少严格线性，但线性回归凭借可解释性、训练速度快、作为更强模型基线（baseline）的优势，至今仍是数据分析与统计建模的首选工具。本章将从最小二乘原理出发，逐步推导出梯度下降、正则化等核心概念，并通过加州房价数据集完成端到端实战。

2.1 算法原理：最小二乘法与 MSE

给定 n 个样本 (x_i, y_i) ，其中 $x_i \in \mathbb{R}^d$ 是 d 维特征向量， $y_i \in \mathbb{R}$ 是目标值。线性回归假设目标与特征呈线性关系：

$$\hat{y}_i = w_1 \cdot x_{i,1} + w_2 \cdot x_{i,2} + \dots + w_d \cdot x_{i,d} + b$$

写成向量形式即 $\hat{y} = Xw + b$ ，其中 $w \in \mathbb{R}^d$ 是权重向量， b 是偏置。**损失函数**采用均方误差（Mean Squared Error, MSE）：

$$L(w, b) = (1/n) \cdot \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

最小二乘法的名字来源于"使误差平方和最小"。求 L 对 w 与 b 的偏导并令其为零，可得**正规方程**（Normal Equation）的解析解：

$$w^* = (X^T X)^{-1} X^T y$$

当特征维度 d 较小且 $X^T X$ 可逆时，解析解一次计算即可得最优参数；但当 d 很大或特征间存在共线性时，矩阵求逆代价高昂（ $O(d^3)$ ）且数值不稳定，此时改用迭代法——**梯度下降**——成为更现实的选择。

2.2 梯度下降法

梯度下降是一种通用的优化算法，核心思想是"沿着损失下降最快的方向走一步"。对 MSE 损失 L ，参数更新规则为：

$$w \leftarrow w - \eta \cdot \partial L / \partial w, b \leftarrow b - \eta \cdot \partial L / \partial b$$

其中 η 是**学习率**（learning rate），控制每步走的距离。学习率过大可能越过极小值甚至发散；学习率过小则收敛缓慢甚至陷入局部极小。实践中常用 learning rate schedule（如余弦退火）或自适应优化器（Adam、Adagrad）动态调整。

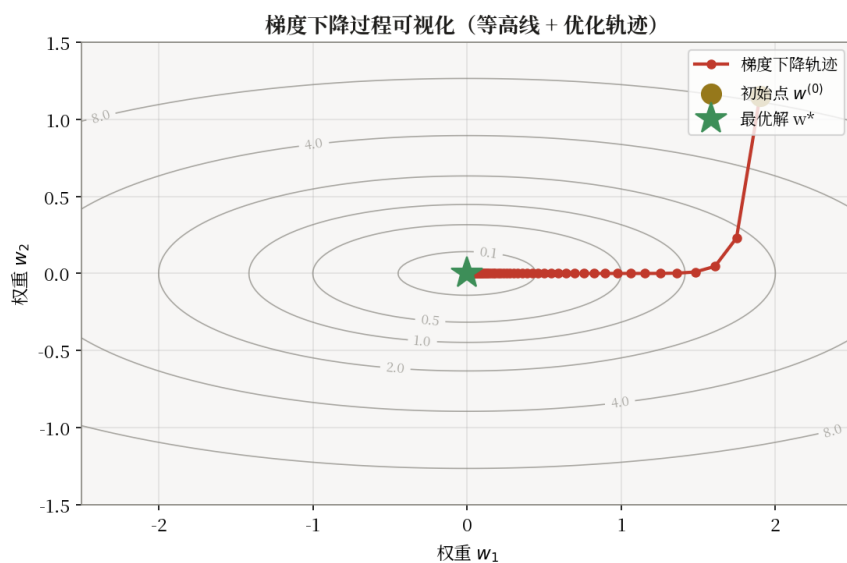


图 2.1 梯度下降示意图：从初始点出发，沿损失曲面最陡方向迭代至谷底

[提示] 当样本量 n 很大时，每次遍历全部样本计算梯度代价过高，可采用**随机梯度下降**（SGD，每次用一个样本）或**小批量梯度下降**（Mini-batch SGD，每次用 32/64/128 个样本）。小批量 SGD 是深度学习训练的事实标准，平衡了梯度估计的稳定性与计算效率。

2.3 正则化：Lasso 与 Ridge

当特征维度高、样本量少时，普通最小二乘解容易过拟合——训练误差很低但泛化能力差。正则化通过在损失函数中加入参数范数惩罚项，约束模型复杂度。两种最常用形式为：

- **L2 正则化（Ridge 岭回归）**：损失中加入 $\lambda \cdot \|w\|^2$ ，使权重整体趋小、平滑但不为 0。适合特征都有贡献、希望模型稳健的场景。
- **L1 正则化（Lasso 回归）**：损失中加入 $\lambda \cdot \|w\|_1$ ，能产生稀疏解，部分权重被压到 0，起到**特征选择**作用。适合高维稀疏数据。

$$L_{\text{Ridge}} = \text{MSE} + \lambda \cdot \sum w_j^2, L_{\text{Lasso}} = \text{MSE} + \lambda \cdot \sum |w_j|$$

其中 $\lambda \geq 0$ 是正则化强度超参数， λ 越大约束越强。实际选择 Ridge 还是 Lasso 取决于问题：若希望模型可解释（保留少数关键特征），用 Lasso；若所有特征都有贡献、目标是稳健预测，用 Ridge。当两者都想要时可用 **ElasticNet**，按比例混合 L1 与 L2 惩罚。

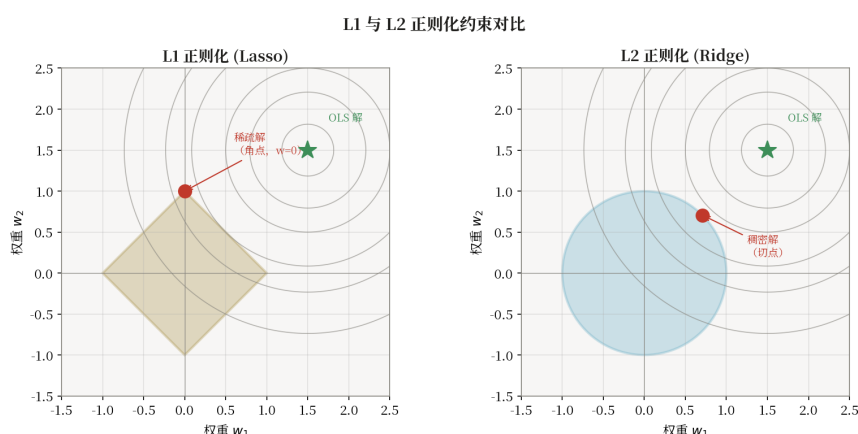


图 2.2 L1 与 L2 正则化几何对比：L1 约束区域为菱形，易在顶点处与损失等高线相交产生稀疏解

2.4 算法实现描述

线性回归的实现核心是两个方法：**fit**（训练）和 **predict**（预测）。训练可用闭式解（正规方程）或梯度下降。以下用伪代码描述。

2.4.1 数据结构

- **权重向量 w** ：长度 d （特征数），模型核心参数
- **偏置 b** ：标量
- **训练数据**： $X \in \mathbb{R}^{n \times d}$, $y \in \mathbb{R}^n$

2.4.2 $\text{fit}(X, y)$ — 梯度下降

输入：特征矩阵 X ，标签向量 y ，学习率 α ，迭代次数 T

步骤：

- 1. 初始化 $w \leftarrow 0, b \leftarrow 0$
- 2. 对 $\text{epoch} = 1, 2, \dots, T$:
 - (a) 计算预测值 $\hat{y} = Xw + b$
 - (b) 计算误差 $\delta = \hat{y} - y$
 - (c) 计算梯度: $\nabla_w = (2/n) \cdot X^T \delta, \nabla_b = (2/n) \cdot \Sigma \delta$
 - (d) 参数更新: $w \leftarrow w - \alpha \cdot \nabla_w, b \leftarrow b - \alpha \cdot \nabla_b$

闭式解（替代方案）： $w^* = (X^T X)^{-1} X^T y$ ，一步求解

2.4.3 predict(X)

$$\hat{y} = Xw + b$$

2.4.4 score(X, y) — R^2

$$R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$$

2.4.5 复杂度

- 闭式解: $O(nd^2 + d^3)$ ，矩阵求逆是瓶颈
- 梯度下降: $O(ndT)$ ， T 为迭代次数
- 预测: $O(nd)$ ，单次矩阵向量乘法

[提示] scikit-learn 的 LinearRegression 用 SVD 分解替代直接求逆，数值更稳定。

2.5 计算示例：三个样本的手算

例题：给定 3 个样本 $(x, y) = \{(1, 2), (2, 4), (3, 5)\}$ ，用最小二乘法拟合简单线性回归 $y_{\text{hat}} = wx + b$ 。

解：先计算统计量：

$$\Sigma x = 1+2+3 = 6, \Sigma y = 2+4+5 = 11, \Sigma x^2 = 1+4+9 = 14, \Sigma xy = 2+8+15 = 25$$

$$\text{样本数 } n = 3, \text{ 均值 } \bar{x} = 2, \bar{y} = 11/3$$

$$w = (n \cdot \Sigma xy - \Sigma x \cdot \Sigma y) / (n \cdot \Sigma x^2 - (\Sigma x)^2)$$

$$= (3 \times 25 - 6 \times 11) / (3 \times 14 - 36)$$

$$= (75 - 66) / (42 - 36) = 9/6 = 1.5$$

$$b = \bar{y} - w \cdot \bar{x} = 11/3 - 1.5 \times 2 = 11/3 - 3 = 2/3 \approx 0.667$$

故拟合直线为 $y_{\text{hat}} = 1.5x + 0.667$ 。代入 $x=1,2,3$ 验证: $y_{\text{hat}} \approx 2.17, 3.67, 5.17$ ，残差平方和 $SSE = (2-2.17)^2 + (4-3.67)^2 + (5-5.17)^2 \approx 0.167$ ，已达最小。

2.6 应用实例：加州房价预测

加州房价数据集 (California Housing) 是 sklearn 内置的经典回归数据集，包含 20640 个街区样本，9 个特征：MedInc（地区收入中位数）、HouseAge、AveRooms、AveBedrms、Population、AveOccup、Latitude、Longitude，目标为 MedHouseVal（房价中位数，单位 10 万美元）。我们用线性回归建立基线模型，再用 Ridge 与 Lasso 对比。

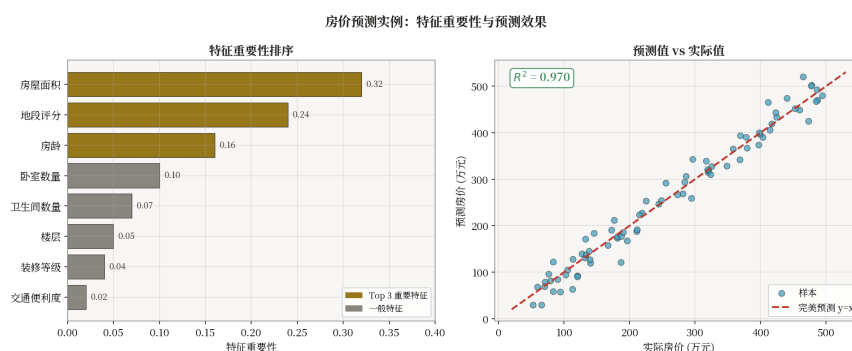


图 2.3 加州房价预测：真实值 vs 预测值散点（左）与特征权重条形图（右）

基线 LinearRegression 测试集 $R^2 \approx 0.60$ ；Ridge ($\lambda=1.0$) $R^2 \approx 0.60$ ，差异不大说明该数据集特征共线性不强；Lasso 会将部分弱相关特征权重压至 0，起到特征筛选作用。若想进一步提升精度，可尝试加入特征交叉（如人均房间数 = AveRooms / AveOccup）、地理聚类特征，或改用梯度提升树 (GBDT) 等更强模型。

2.7 代码实现：sklearn LinearRegression

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score

# — 加载加州房价数据集 —
X, y = fetch_california_housing(return_X_y=True)
print(f"样本数: {X.shape[0]}, 特征数: {X.shape[1]}")

# — 划分训练集与测试集 (8:2, 固定随机种子) —
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
# — 特征标准化（必须用训练集统计量 fit，避免数据泄漏） —
scaler = StandardScaler().fit(X_tr)
X_tr_s = scaler.transform(X_tr)
X_te_s = scaler.transform(X_te)

# — 训练三个模型：普通 / Ridge / Lasso —
models = {
    "LinearRegression": LinearRegression(),
    "Ridge(a=1.0)": Ridge(alpha=1.0),
    "Lasso(a=0.1)": Lasso(alpha=0.1),
}
for name, m in models.items():
    m.fit(X_tr_s, y_tr)
    y_pred = m.predict(X_te_s)
    rmse = np.sqrt(mean_squared_error(y_te, y_pred))
    r2 = r2_score(y_te, y_pred)
    print(f"{name:20s} RMSE={rmse:.3f}, R2={r2:.3f}")

# — 查看 Lasso 权重（部分会被压为 0） —
print("Lasso 权重:", models["Lasso(a=0.1)"].coef_)
```

2.8 现代发展

线性回归看似简单，但其思想贯穿整个机器学习发展史。现代演化包括：

- **广义线性模型 GLM**：将线性回归扩展到分类（逻辑回归）、泊松回归、Gamma 回归等，通过链接函数 $g(\mu) = Xw$ 连接线性预测与分布参数。
- **自动化特征工程**：Featuretools、AutoFeat 等工具自动构造交叉、聚合、时序特征，降低手工成本。
- **自动化超参调优**：Optuna、Hyperopt、sklearn GridSearchCV 等，结合贝叶斯优化、Hyperband 实现高效调参。
- **正则化进化**：ElasticNet（L1+L2 混合）、Group Lasso（组稀疏）、fused Lasso（结构稀疏）满足不同约束需求。
- **可解释性方法**：SHAP、LIME 等方法将任意黑箱模型“翻译”为线性加权形式，弥合精度与可解释性的鸿沟。

第三章 决策树

贷款审批是金融机构每天面对的核心决策：是否给一位申请人发放贷款？传统做法是基于风控专家经验制定规则——“月薪 > 1 万且征信评分 > 700 才批”，但规则过多就难以维护，且无法从历史数据中学习最优划分。决策树（Decision Tree）正是为这类**可解释性优先**的场景而生：它从训练数据中自动归纳出一组 if-else 规则，形如一棵倒置的树，每条从根到叶的路径就是一条可读的决策规则。相比于线性回归“全局线性”假设，决策树天然能捕捉非线性关系与特征交互，同时保持白盒可解释性，是金融风控、医疗诊断、营销响应等领域的常青算法。本章将系统介绍 ID3/C4.5/CART 三大经典算法、剪枝策略、计算示例与代码实现。

3.1 算法原理：三大经典算法

决策树的核心问题是：在每个节点如何选择最优分裂特征？不同分裂准则衍生出三大经典算法。

3.1.1 ID3：信息增益

ID3 由 Quinlan 于 1986 年提出，以**信息增益**作为分裂准则。它基于香农信息熵：数据集 D 的熵 $H(D)$ 度量其类别不确定性：

$$H(D) = -\sum_{k=1}^K p_k \cdot \log_2 p_k$$

按特征 A 分裂后的**条件熵**与**信息增益**定义为：

$$H(D|A) = \sum_v (|D_v|/|D|) \cdot H(D_v), g(D, A) = H(D) - H(D|A)$$

ID3 选择使信息增益最大的特征作为当前节点的分裂特征。缺点是偏向取值多的特征（极端如 ID 这种高基数特征会获得最大增益但毫无意义）。

3.1.2 C4.5：增益率

C4.5 用**增益率**（Gain Ratio）修正 ID3 的偏向，引入"分裂信息"度量特征自身取值的均匀度：

$$\text{GainRatio}(D, A) = g(D, A) / \text{SplitInfo}(A), \text{SplitInfo}(A) = -\sum_v (|D_v|/|D|) \cdot \log_2(|D_v|/|D|)$$

C4.5 还支持处理连续特征（通过二分离散化）、缺失值与剪枝，是 ID3 的全面升级版。

3.1.3 CART：基尼系数

CART（Classification and Regression Tree）由 Breiman 于 1984 年提出，采用**基尼系数**作为分类准则，避免对数计算更高效：

$$\text{Gini}(D) = 1 - \sum_{k=1}^K p_k^2, \text{Gini}(D, A) = \sum_v (|D_v|/|D|) \cdot \text{Gini}(D_v)$$

CART 与 ID3/C4.5 的最大区别是**只生成二叉树**（每个节点只做二分裂），并且既可做分类（基尼系数）也可做回归（用平方误差作为分裂准则）。sklearn 中的 `DecisionTreeClassifier` 即基于 CART 实现。

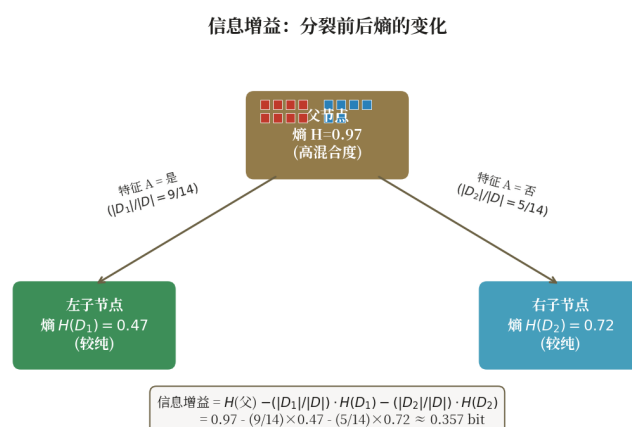


图 3.1 信息增益示意：分裂前父节点熵高，分裂后子节点熵低，差值即信息增益

3.2 决策树结构

一棵决策树由**根节点**、**内部节点**、**叶子节点**与**边**构成。根节点代表整个数据集；每个内部节点对应一次特征分裂测试（如“年龄 < 30?”）；每条边代表测试结果的一个分支；叶子节点携带最终预测值——分类时是类别标签或概率分布，回归时是叶子内样本目标均值。从根到任一叶子节点的路径就是一条决策规则，例如“年龄<30 且 收入>1万 → 批准贷款”。正是这种规则可读性使决策树在合规审计要求严格的行业广受欢迎。

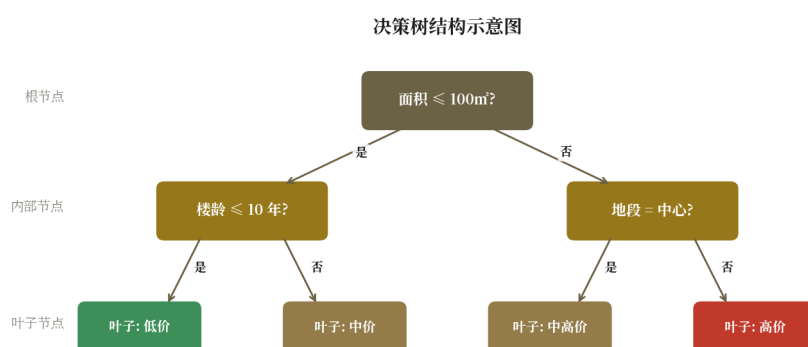


图 3.2 决策树结构示意：根节点 → 内部节点 → 叶子节点

3.3 剪枝：预剪枝 vs 后剪枝

不限制生长的决策树会一直分裂到每个叶子节点纯度为 0，极易过拟合——训练集准确率 100% 但测试集表现差。**剪枝**是控制决策树复杂度的核心手段，分两种：

- **预剪枝 (Pre-pruning)**：在生长过程中提前停止。常用停止条件：max_depth（最大深度）、min_samples_split（节点最少样本数）、min_samples_leaf（叶子最少样本数）、max_leaf_nodes。优点是简单高效，缺点是可能欠拟合。
- **后剪枝 (Post-pruning)**：先让树充分生长，再自底向上考察每个内部节点：若将其替换为叶子节点能提升验证集性能，则剪掉。代表方法：Cost-Complexity Pruning（CART 的 ccp_alpha 参数）。优点是精度通常更高，缺点是计算开销大。

[技巧] sklearn 同时支持预剪枝（max_depth 等参数）与后剪枝（ccp_alpha 参数），实践中常先用 max_depth 控制规模，再用 GridSearchCV 调 ccp_alpha 微调。

3.4 算法实现描述

决策树的核心是**递归分裂**：每个节点选择最优特征和分裂点，将数据分为两个子集，对子集递归建树。需要定义树节点数据结构和分裂准则。

3.4.1 数据结构

- **TreeNode**: { feature (分裂特征索引), threshold (分裂阈值), left (左子树), right (右子树), label (叶节点类别) }
- 参数: max_depth (最大深度), min_samples_split (最小分裂样本数)

3.4.2 fit(X, y) — 递归建树

输入: 特征矩阵 X, 标签 y, 当前深度 depth

递归过程:

- 1. 若满足停止条件 (纯度足够/深度达限/样本太少) → 创建叶节点, label = 众数(y)
- 2. 否则, 遍历每个特征 j 和每个候选阈值 t:
 - (a) 按阈值分割: 左集 = $\{(x,y) \mid x_j \leq t\}$, 右集 = 其余
 - (b) 计算分裂后基尼系数: $G = (|左|/n) \cdot \text{Gini}(左) + (|右|/n) \cdot \text{Gini}(右)$
 - (c) 选取使 G 最小的 (j^*, t^*)
- 3. 创建内部节点, 存储 (j^*, t^*) , 递归调用 fit(左集, depth+1) 和 fit(右集, depth+1)

3.4.3 基尼系数

$$\text{Gini}(S) = 1 - \sum_k p_k^2, \text{ 其中 } p_k = |S_k| / |S|$$

3.4.4 predict(X) — 树遍历

对每个样本 x: 从根节点出发, 按 $x_{\text{feature}} \leq \text{threshold}$ 走左/右子树, 到达叶节点返回 label。

3.4.5 复杂度

- 训练: $O(n \cdot d \cdot \log n)$ 每层, 共 $O(n \cdot d \cdot \log^2 n)$
- 预测: $O(\log n)$ 单样本, 树深度为 $O(\log n)$

[提示] scikit-learn 的 CART 实现用排序+前缀和加速分裂搜索, 比朴素遍历快数倍。

3.5 计算示例: 用信息增益选分裂特征

例题: 下表是 6 位贷款申请人的简化数据, 类别 Y=批准 / N=拒绝, 问应优先用哪个特征分裂?

ID	有工作	信用好	类别
1	否	否	N
2	否	是	N
3	是	是	Y
4	是	否	Y
5	否	是	N
6	是	是	Y

解: 根节点 D 共 6 个样本, 3 Y 3 N, 熵 $H(D) = 1 \text{ bit}$ 。

按"有工作"分裂：是 $\rightarrow\{3,4,6\}$ 全 Y，熵 0；否 $\rightarrow\{1,2,5\}$ 全 N，熵 0。

$H(D|\text{有工作}) = (3/6) \times 0 + (3/6) \times 0 = 0$ ，信息增益 $= 1 - 0 = 1 \text{ bit}$ 。

按"信用好"分裂：是 $\rightarrow\{2,3,5,6\} = \{N,Y,N,Y\}$ ，熵 1 bit；否 $\rightarrow\{1,4\} = \{N,Y\}$ ，熵 1 bit。

$H(D|\text{信用好}) = (4/6) \times 1 + (2/6) \times 1 = 1 \text{ bit}$ ，信息增益 $= 1 - 1 = 0 \text{ bit}$ 。

结论："有工作"信息增益（1 bit）远大于"信用好"（0 bit），应优先按"有工作"分裂。分裂后子节点已纯，停止生长，得一棵只有根节点的简单决策树。

3.6 应用实例：贷款审批决策树

某银行历史贷款数据集包含 1000 条记录，特征含年龄、收入、职业、负债率、征信评分、贷款金额、期限等 12 个字段，目标为是否违约。使用 sklearn DecisionTreeClassifier 训练并设置 max_depth=5 以保证可解释性，得到的决策树典型规则如：

- "负债率 $> 35\%$ 且 征信评分 $< 680 \rightarrow$ 拒绝"（高风险路径）
- "负债率 $\leq 35\%$ 且 月收入 $> 8000 \rightarrow$ 批准"（低风险路径）
- "负债率 $\leq 35\%$ 且 月收入 ≤ 8000 且 工龄 < 2 年 $\rightarrow$ 拒绝"

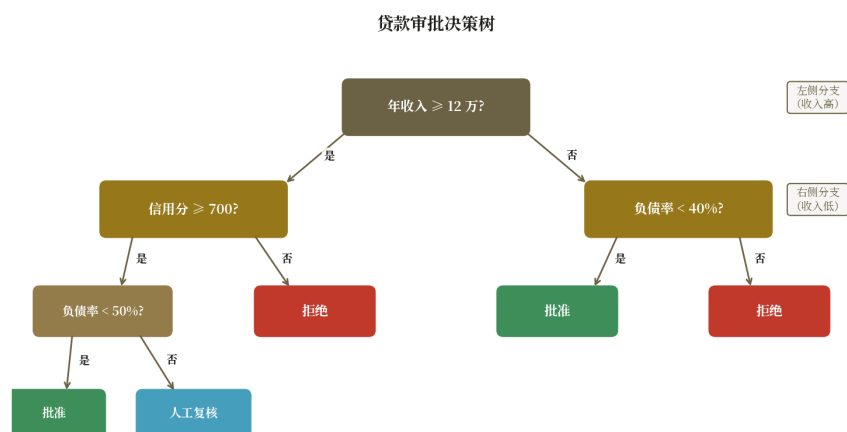


图 3.3 贷款审批决策树：根节点为负债率，深色叶子代表批准、浅色代表拒绝

该树在测试集上准确率约 82%，AUC 0.78。若需进一步提升性能，可将多棵决策树组合为集成模型——随机森林（Bagging）、GBDT/XGBoost/LightGBM（Boosting）——通常能将 AUC 提升到 0.85+，代价是牺牲可解释性。

3.7 代码实现：sklearn DecisionTreeClassifier

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

# — 加载鸢尾花数据集 —
X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# — 训练决策树 (CART, 基尼系数, max_depth=3) —
clf = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    min_samples_leaf=5,
    random_state=42,
)
clf.fit(X_tr, y_tr)

# — 评估 —
y_pred = clf.predict(X_te)
print(f"测试集准确率: {accuracy_score(y_te, y_pred):.3f}")
print(classification_report(y_te, y_pred,
    target_names=load_iris().target_names))

# — 可视化决策树 —
plt.figure(figsize=(12, 8))
plot_tree(clf, feature_names=load_iris().feature_names,
    class_names=load_iris().target_names,
    filled=True, rounded=True)
plt.title("Iris Decision Tree (max_depth=3)")
plt.tight_layout()
plt.savefig("iris_tree.png", dpi=120)

# — 查看特征重要性 —
for name, imp in zip(load_iris().feature_names,
    clf.feature_importances_):
    print(f"{name:20s} importance={imp:.3f}")
```

3.8 现代发展

单棵决策树虽然可解释，但精度受限。现代工业界的核心发展方向是**集成学习**：

- **随机森林 (Random Forest)**：Bagging 思想，训练多棵独立决策树，投票/平均输出。降低方差，抗过拟合，开箱即用。
- **GBDT**：Boosting 思想，每棵新树拟合前一棵的残差（负梯度）。代表实现：sklearn GradientBoostingClassifier、XGBoost、LightGBM、CatBoost。
- **XGBoost / LightGBM / CatBoost**：第三代梯度提升框架，引入二阶导数、直方图近似、Leaf-wise 生长、GPU 加速等优化，长期统治 Kaggle 表格数据竞赛。
- **可解释 Boosting**：Interpretable Boosting Machine (EBM/GAM) 保留决策树可解释性，精度接近 GBDT。
- **软决策树**：用神经网络模拟决策树，支持端到端反向传播训练，是树模型与深度学习的融合方向。

第四章 聚类分析

客户分群是营销与运营最经典的诉求：电商希望把用户分为"价格敏感型"、"品牌忠诚型"、"冲动消费型"等群体，针对性地推送优惠与商品。但与分类不同——客户通常没有现成的标签，我们手中只有大量行为日志（点击、购买、停留时长）。如何在无标签数据中发现自然群体结构？这正是**聚类分析**要解决的问题。聚类属于无监督学习，目标是将样本划分为若干组，使组内相似度高、组间相似度低。它在客户分群、异常检测、图像分割、文档主题发现等场景中应用广泛。本章将介绍 K-Means、DBSCAN、层次聚类三大算法，并讨论评估指标与工程实战。

4.1 K-Means 原理

K-Means 是最经典的**划分式聚类**算法，1957 年由 Lloyd 提出。它要求预先指定簇数 K ，通过反复迭代"分配-更新"两个步骤收敛到局部最优：

- **1. 初始化**：随机选择 K 个样本作为初始簇中心（centroid），或用 K-Means++ 智能初始化以加速收敛。
- **2. 分配**：将每个样本分配到最近的簇中心，形成 K 个簇。距离常用欧氏距离： $d(x, c) = \|x - c\|_2$ 。
- **3. 更新**：对每个簇重新计算簇内样本均值作为新簇中心。
- **4. 收敛判断**：若簇中心移动量小于阈值或达到最大迭代数则停止，否则回到步骤 2。

$$J = \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|^2$$

K-Means 实际上是在最小化上式目标函数 J （簇内平方误差和 SSE），但该问题为 NP-hard，迭代算法只能保证收敛到局部最优，因此实践中常多次随机初始化取最优结果（`n_init` 参数）。

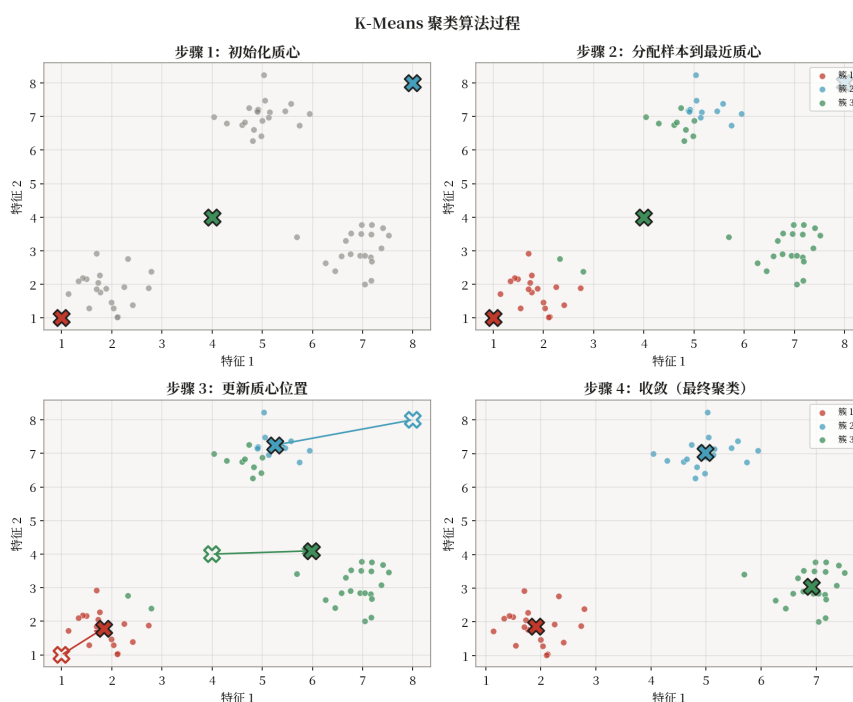


图 4.1 K-Means 迭代过程：初始化 → 分配 → 更新 → 收敛

4.2 DBSCAN 原理

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) 是经典的密度式聚类算法, 1996 年由 Ester 等人提出。与 K-Means 不同, DBSCAN 不需要预先指定簇数, 且能识别任意形状的簇与噪声点。它基于两个参数: ϵ (邻域半径) 和 MinPts (核心点最小邻居数)。每个点被分为三类:

- **核心点 (Core Point)**: ϵ 邻域内样本数 $\geq$ MinPts。位于稠密区域中心。
- **边界点 (Border Point)**: 落在某核心点的 ϵ 邻域内, 但自身邻域样本数 $<$ MinPts。位于簇边缘。
- **噪声点 (Noise Point)**: 既非核心也非边界, 被视为离群点。

算法从任一未访问点出发: 若该点是核心点, 则其 ϵ -邻域内的所有点 (含核心点邻居的核心点的邻居, 递归) 构成一个簇; 若该点是噪声, 则标记后跳过。DBSCAN 的最大优势是能识别**非凸形状** (如两个月牙形簇) 的聚类, 且天然具备异常检测能力——噪声点就是异常候选。

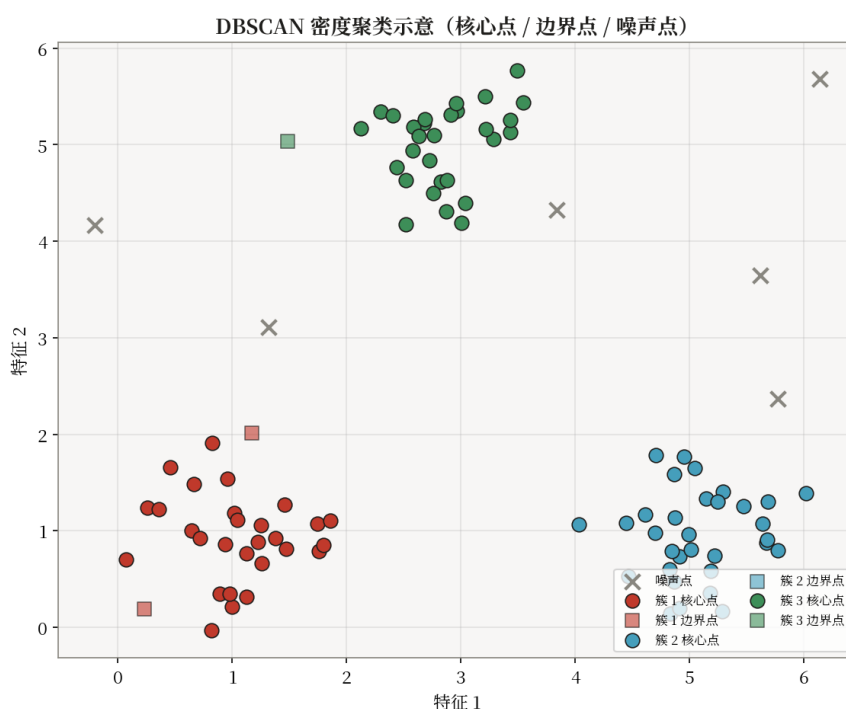


图 4.2 DBSCAN 聚类示意: 核心点稠密成簇, 边界点环绕, 噪声点散落

4.3 层次聚类

层次聚类 (Hierarchical Clustering) 无需指定 K, 而是构建一棵聚类树 (dendrogram)。依据构建方向分为两种策略:

- **自底向上 (聚合型 Agglomerative)**: 初始时每个样本自成一类, 每次合并最相似的两类, 直到全部合并为一类。最常用。
- **自顶向下 (分裂型 Divisive)**: 初始时所有样本为一类, 递归地将每类分裂为两子类, 直到每类只剩一个样本。计算量更大, 少用。

聚合型层次聚类的关键在于**簇间距离** (linkage) 定义: single linkage (最近邻距离)、complete linkage (最远邻距离)、average linkage (平均距离)、ward linkage (基于平

方误差增量)。构建好 dendrogram 后, 通过设定距离阈值或目标簇数即可"剪开"得到具体聚类。

4.4 评估指标

聚类没有真实标签, 评估难度高于分类。常用内部指标如下:

- **轮廓系数 (Silhouette Coefficient)**: 对每个样本计算 $s = (b-a)/\max(a,b)$, 其中 a 是同簇平均距离, b 是到最近其他簇的平均距离。取值 $[-1, 1]$, 越大越好, >0.5 表示聚类合理。
- **DB 指数 (Davies-Bouldin Index)**: 衡量簇内紧密度与簇间分离度的比值, 越小越好。
- **CH 指数 (Calinski-Harabasz)**: 簇间离散度与簇内离散度之比, 越大越好。
- **SSE / 拐点法**: 画出 SSE 随 K 变化曲线, 找"肘部"作为最佳 K 。

[提示] 当有真实标签 (如做对比实验) 时, 可用外部指标: ARI (Adjusted Rand Index)、NMI (归一化互信息)、FMI (Fowlkes-Mallows Index), 取值 $[0, 1]$, 越大越好。

4.5 算法实现描述

K-Means 实现包含 fit (训练)、predict (分配簇)、transform (距离矩阵) 三个方法。核心是 Lloyd 迭代: 初始化中心 $\rightarrow$ 分配 $\rightarrow$ 更新 $\rightarrow$ 判断收敛。

4.5.1 数据结构

- **cluster_centers_**: 矩阵 $[k, d]$, k 个簇中心坐标
- **labels_**: 向量 $[n]$, 每个样本的簇分配
- **inertia_**: 标量, 簇内方差和 (收敛指标)

4.5.2 fit(X) — Lloyd 迭代

输入: 数据 $X \in \mathbb{R}^{n \times d}$, 簇数 k , 最大迭代 T

步骤:

- 1. **K-Means++ 初始化**: 随机选第 1 个中心; 对 $i = 2, \dots, k$: 计算各点到已选中心的最小距离 $D(x)$, 按概率 $P(x) \propto D(x)^2$ 选下一个中心
- 2. **迭代** ($t = 1, \dots, T$):
 - (a) **分配步**: 对每个 x_i , $\text{labels}_i \leftarrow \arg\min_j \|x_i - c_j\|^2$
 - (b) **更新步**: 对每个簇 j , $c_j \leftarrow \text{mean}(\{x_i \mid \text{labels}_i = j\})$
 - (c) 若中心不变则收敛, 提前终止
- 3. 计算 $\text{inertia}_t = \sum_i \|x_i - c_{\text{labels}_i}\|^2$

4.5.3 predict(X)

$$\text{label}(x) = \arg\min_j \|x - c_j\|^2$$

4.5.4 transform(X)

返回 $[n, k]$ 距离矩阵 $D_{ij} = \|x_i - c_j\|$

4.5.5 复杂度

- 训练: $O(nkdT)$, T 为迭代次数
- 预测: $O(nkd)$, 计算到 k 个中心的距离

[提示] scikit-learn 用 Elkan 三角不等式加速, 避免部分距离计算, 比朴素实现快 2-10 倍。

4.6 计算示例: 5 个点的 K-Means 手算

例题: 5 个一维样本 $X = \{1, 2, 8, 9, 25\}$, 设 $K=2$, 初始中心 $c_1=1, c_2=2$, 做 K-Means 迭代直到收敛。

解: 第 1 轮分配 (按到中心距离) :

$1 \rightarrow C_1 (|1-1|=0)$; $2 \rightarrow C_2 (|2-2|=0)$; $8 \rightarrow C_2 (6 \text{ vs } 7)$; $9 \rightarrow C_2 (7 \text{ vs } 8)$; $25 \rightarrow C_2 (23 \text{ vs } 24)$

得 $C_1=\{1\}$, $C_2=\{2,8,9,25\}$

第 1 轮更新: $c_1=1, c_2=\text{mean}(2,8,9,25)=44/4=11$

第 2 轮分配 ($c_1=1, c_2=11$) :

$1 \rightarrow C_1 (0 \text{ vs } 10)$; $2 \rightarrow C_1 (1 \text{ vs } 9)$; $8 \rightarrow C_2 (7 \text{ vs } 3)$; $9 \rightarrow C_2 (8 \text{ vs } 2)$; $25 \rightarrow C_2 (24 \text{ vs } 14)$

得 $C_1=\{1,2\}$, $C_2=\{8,9,25\}$

第 2 轮更新: $c_1=\text{mean}(1,2)=1.5, c_2=\text{mean}(8,9,25)=42/3=14$

第 3 轮分配 ($c_1=1.5, c_2=14$) :

$1 \rightarrow C_1 (0.5 \text{ vs } 12.5)$; $2 \rightarrow C_1 (0.5 \text{ vs } 11.5)$; $8 \rightarrow C_2 (6.5 \text{ vs } 6)$; $9 \rightarrow C_2 (7.5 \text{ vs } 5)$; $25 \rightarrow C_2 (23.5 \text{ vs } 11)$

得 $C_1=\{1,2\}$, $C_2=\{8,9,25\}$ (与上轮相同, 已收敛)

最终聚类 $C_1=\{1,2\}$, $C_2=\{8,9,25\}$, $SSE = (1-1.5)^2 + (2-1.5)^2 + (8-14)^2 + (9-14)^2 + (25-14)^2 = 0.25 + 0.25 + 36 + 25 + 121 = 182.5$ 。

注意: 若初始中心改为 $c_1=1, c_2=25$, 会收敛到 $C_1=\{1,2,8,9\}$, $C_2=\{25\}$, SSE 仅 50——这正说明 K-Means 对初始中心敏感, 需多次初始化取最优。

4.7 应用实例 1: 客户 RFM 分群

RFM 是营销领域最常用的客户价值模型: R (Recency, 最近一次购买距今天数)、F (Frequency, 购买次数)、M (Monetary, 累计消费金额)。某电商平台 10 万活跃用户, 对 (R, F, M) 三维特征先做对数变换 + 标准化, 用 K-Means 聚为 4 簇, 业务团队依据各簇均值打

标签：

- 簇 1：高价值忠诚客（R 低、F 高、M 高，约 15%）——重点维系，主推会员升级。
- 簇 2：潜力唤醒客（R 高、F 中、M 中，约 25%）——发优惠券刺激复购。
- 簇 3：低频大额客（R 高、F 低、M 高，约 10%）——主推新品，提升频次。
- 簇 4：边缘流失客（R 高、F 低、M 低，约 50%）——低成本触达或放弃。

客户分群结果（RFM 三维可视化）

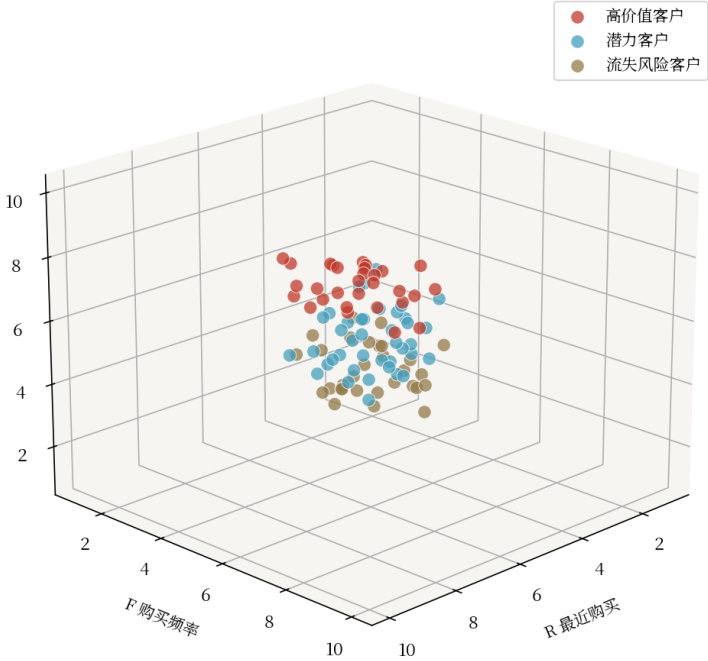


图 4.3 客户 RFM 分群结果：4 个簇在 R-F-M 三维空间中的分布

4.8 应用实例 2：图像颜色量化压缩

颜色量化是图像压缩的经典技巧：一张照片可能含 16 万种颜色，若压缩到 16 色仍能保留视觉质量，文件体积可减少 80% 以上。做法是把所有像素 R-G-B 三维向量视为样本，用 K-Means 聚为 K 类，再用每类的簇中心颜色替换原像素颜色。sklearn 提供了直接可用的接口：cluster.KMeans 对 (H×W, 3) 像素矩阵聚类，centers 即调色板，labels 即索引图。

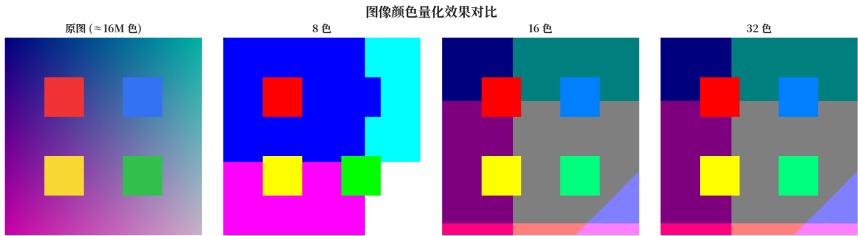


图 4.4 颜色量化效果对比：原图（24 位真彩色）与 K=16 量化后的压缩图

4.9 应用实例 3：异常检测

聚类算法天然能识别异常点。常用思路：

- **K-Means 距离法**：计算每个样本到其簇中心的距离，距离大于阈值（如 95 分位数）的视为异常。
 - **DBSCAN 噪声法**：直接利用 DBSCAN 标记的噪声点作为异常，无需调阈值。
 - **孤立森林 (Isolation Forest)**：专门为异常检测设计的树模型，利用随机划分路径长度判定异常，效率极高。
- 典型场景包括信用卡欺诈检测、网络入侵检测、工业设备故障预警、医疗指标异常监控等。

4.10 代码实现：sklearn KMeans 与 DBSCAN

```
import numpy as np
from sklearn.cluster import KMeans, DBSCAN
from sklearn.datasets import make_moons, make_blobs
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# — 生成数据: 300 个 blob 样本 + 200 个月牙样本 —
X_blob, _ = make_blobs(n_samples=300, centers=4,
                        cluster_std=0.6, random_state=42)
X_moon, _ = make_moons(n_samples=200, noise=0.08,
                        random_state=42)

# — K-Means: 必须先标准化 —
X_s = StandardScaler().fit_transform(X_blob)
km = KMeans(n_clusters=4, init="k-means++",
            n_init=10, random_state=42)
labels_km = km.fit_predict(X_s)
print(f"K-Means 轮廓系数: "
      f"{silhouette_score(X_s, labels_km):.3f}")

# — DBSCAN: 擅长非凸形状, 无需指定 K —
X_m = StandardScaler().fit_transform(X_moon)
db = DBSCAN(eps=0.3, min_samples=5)
labels_db = db.fit_predict(X_m)
n_clusters = len(set(labels_db)) - (1 if -1 in labels_db else 0)
n_noise = list(labels_db).count(-1)
print(f"DBSCAN 发现 {n_clusters} 个簇, 噪声点 {n_noise} 个")

# — 用肘部法确定最佳 K —
sse = []
for n in range(1, 11):
    km_n = KMeans(n_clusters=n, n_init=10,
                  random_state=42).fit(X_s)
    sse.append(km_n.inertia_)
print("SSE 曲线:", [round(s, 1) for s in sse])
# 肘部位置对应最佳 K, 结合轮廓系数验证
```

4.11 现代发展

聚类研究仍在快速演进，几个值得关注的方向：

- **深度聚类 (Deep Clustering)**：用深度网络（如 AutoEncoder）先学表示再聚类，代表方法 DEC、DeepCluster、SCAN。在图像、文本等高维数据上效果远超传统 K-Means。
- **谱聚类 (Spectral Clustering)**：基于图拉普拉斯矩阵特征向量做聚类，能识别任意形状的簇，常用于社交网络社区发现。

- **HDBSCAN**: DBSCAN 的层次扩展，自动选择密度阈值，对参数不敏感，已成为密度聚类新标准。
- **主题模型**: LDA、BERTopic 等本质上是文本数据上的软聚类，用于文档主题发现。
- **对比聚类**: 结合对比学习与聚类的自监督方法（如 SwAV、MoCo-cluster），是自监督表示学习与聚类融合的前沿方向。

第五章 集成学习

在前几章里，我们接触了线性回归、决策树、聚类等单一模型。单一模型虽然简单直观，但往往受限于自身的偏差-方差权衡：线性模型偏差大、方差小；深度决策树方差大、偏差小。能否让多个"弱学习器"协作，把各自的优点叠加、缺点相互抵消？这正是**集成学习**（Ensemble Learning）的核心思想。从 Kaggle 比赛到工业界排序、风控系统，集成方法几乎垄断了表格类数据的最佳成绩。本章将系统介绍 Bagging、Boosting、Stacking 三大范式，并以随机森林和梯度提升树为主线展开。

5.1 三大集成范式

集成学习按基学习器的生成方式与组合方式，可分为三大范式：**Bagging**（Bootstrap Aggregating）通过有放回采样并行训练多个独立模型，再以投票或平均方式组合，主要降低方差；**Boosting** 以串行方式训练，每个新模型聚焦于前一轮的错误样本，逐步减小偏差；**Stacking**（Stacked Generalization）则用异构基模型并行训练，再训练一个"元学习器"学习如何组合各基模型的输出。

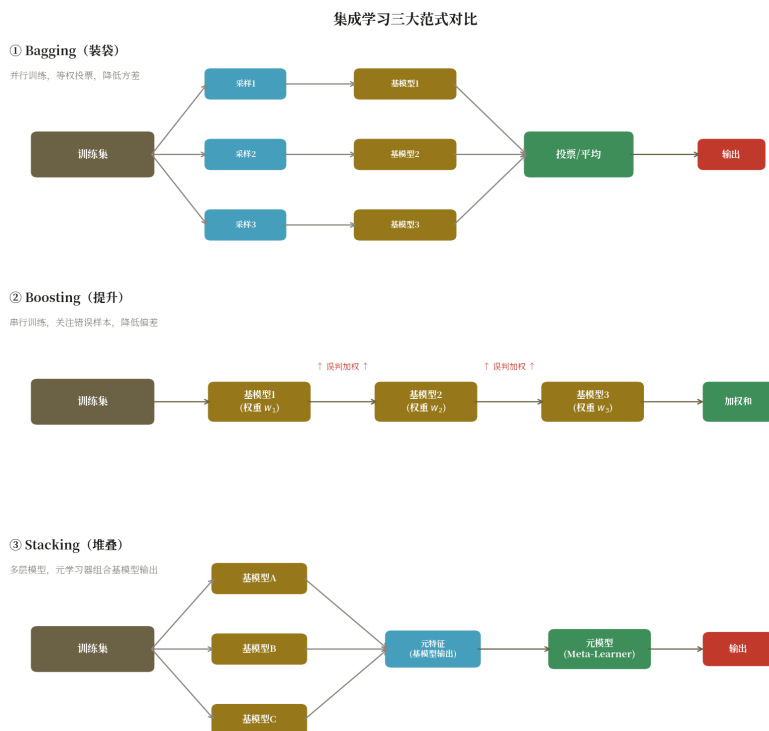


图 5.1 三大集成范式对比：Bagging 并行、Boosting 串行、Stacking 分层

直觉理解：Bagging 像"找一群同等水平的同事分别独立做题再投票"，Boosting 像"先做错题、再让下一个同学专攻错题"，Stacking 则像"让不同专长的专家先各自打分，再由主席加权汇总"。不同的协作方式决定了它们适用的问题类型：Bagging 适合高方差模型（如深树），Boosting 适合高偏差场景，Stacking 适合多源异构模型。

5.2 Bagging 与随机森林

Bagging 的关键是**自助采样** (bootstrap)：从 N 个样本中有放回地抽取 N 次，约 63.2% 的样本会被选中，其余 36.8% 为"袋外样本" (OOB)，可直接用作验证集。在每一份采样数据上独立训练一棵决策树，最终对所有树的预测取平均（回归）或多数投票（分类）。由于每棵树都深达叶节点且未经剪枝，单树方差极大；但相互独立的多树平均后，方差被显著降低。

随机森林 (Random Forest, RF) 在 Bagging 基础上引入**特征随机**：每个节点分裂时，只在随机抽取的 m 个特征（通常 $m = \sqrt{d}$ ）中选择最优分裂。这一改动让树与树之间进一步"去相关"，避免少数强特征反复出现在所有树的根节点，从而让 RF 在大多数表格任务上成为强基线 (strong baseline)。

5.3 Boosting 与梯度提升树

Boosting 的思想源自 Valiant 与 Kearns 的"弱学习提升"问题：能否把一组略好于随机猜测的弱学习器组合成强学习器？AdaBoost (1995) 以样本权重为媒介串行训练，每轮提升错分样本权重，并给准确率高的弱学习器更大投票权。其训练目标可视为最小化指数损失。

梯度提升树 (Gradient Boosted Decision Tree, GBDT) 将 Boosting 写成梯度下降形式：设当前模型为 $F(x)$ ，损失为 $L(y, F(x))$ ，则下一棵树拟合的"目标"是损失对 F 的负梯度（也称伪残差） $r = -\partial L / \partial F$ 。每加入一棵树都让总损失沿梯度方向下降一步，因此可以替换任意可微损失，用于回归、分类、排序、生存分析等多种任务。

$$F_m(x) = F_{m-1}(x) + v \cdot h_m(x), h_m \approx -\partial L / \partial F$$

其中 v 是学习率 (shrinkage)，用于控制每棵树的贡献、防过拟合。**XGBoost** 在 GBDT 基础上引入二阶泰勒展开（用一阶+二阶导数）、正则化项、列抽样与并行分裂查找，显著加速并提升精度；**LightGBM** 采用直方图算法把连续特征离散化到桶中，复杂度从 $O(n \cdot d)$ 降到 $O(b \cdot d)$ ，并采用叶子优先 (leaf-wise) 生长策略，在稀疏特征与大数据上速度领先。

5.4 算法实现描述

随机森林 = Bagging + 决策树 + 特征随机。对每棵树独立采样训练，预测时多数投票。

5.4.1 数据结构

- **trees**: 列表，存 T 棵决策树
- **feature_indices**: 每棵树使用的特征子集
- **参数**: `n_estimators` (树数), `max_depth`, `max_features` (特征子集大小)

5.4.2 `fit(X, y)` — 并行训练

输入: 数据 (X, y) , 树数 T , 特征子集大小 m (通常 $\sqrt{d}$)

步骤 (对 $t = 1, \dots, T$ 并行)：

- 1. **Bootstrap 采样**: 从 n 个样本中有放回随机抽取 n 个 (允许重复)
- 2. **特征随机**: 从 d 个特征中随机选 m 个 ($m \approx \sqrt{d}$)

- 3. 在采样子集上训练一棵决策树（不剪枝）
- 4. 保存树和特征索引

5.4.3 predict(X) — 多数投票

对每个样本 x ：让 T 棵树各自预测，取众数作为最终结果：

$$\hat{y} = \text{mode}(\{h_t(x)\}_{t=1}^T)$$

5.4.4 复杂度

- 训练： $O(T \cdot n \cdot d \cdot \log n)$ ，可完全并行
- 预测： $O(T \cdot \log n)$ ， T 棵树并行投票

[提示] scikit-learn 用 joblib 并行训练，`n_jobs=-1` 利用全部 CPU。XGBoost 用直方图加速。

5.5 计算示例：简单 Bagging 投票手算

假设训练集有 5 个二分类样本 $y = [1, 1, 1, 0, 0]$ ，三棵决策树分别做有放回采样得到不同子集：T1 用 $\{1, 2, 3, 3, 4\}$ 预测结果 $[1, 1, 1, 1, 0]$ ；T2 用 $\{2, 2, 3, 5, 5\}$ 预测 $[1, 1, 1, 0, 0]$ ；T3 用 $\{1, 1, 4, 4, 5\}$ 预测 $[1, 1, 0, 0, 0]$ 。对样本 3，三棵树预测分别为 1、1、0，多数投票为 1，与真实标签一致；对样本 4，三棵树预测为 1、0、0，多数投票为 0，与真实标签一致。可以看到，尽管单棵树对样本 4 容易预测为 1（错误），投票后系统纠正了错误，这正是 Bagging 降低方差的直观体现。

5.6 应用实例：特征重要性分析

在金融风控、医疗诊断等领域，模型不仅要“准”，还要“可解释”——业务方需要知道哪些变量真正驱动了预测。随机森林与 GBDT 都能输出**特征重要性**，常用度量有两种：基于不纯度下降（MDI）和基于置换精度（MDA）。前者累计每个特征在所有树中分裂带来的不纯度减少，速度快但偏向高基数特征；后者打乱某特征的取值后观察模型性能下降幅度，更稳健但计算更贵。

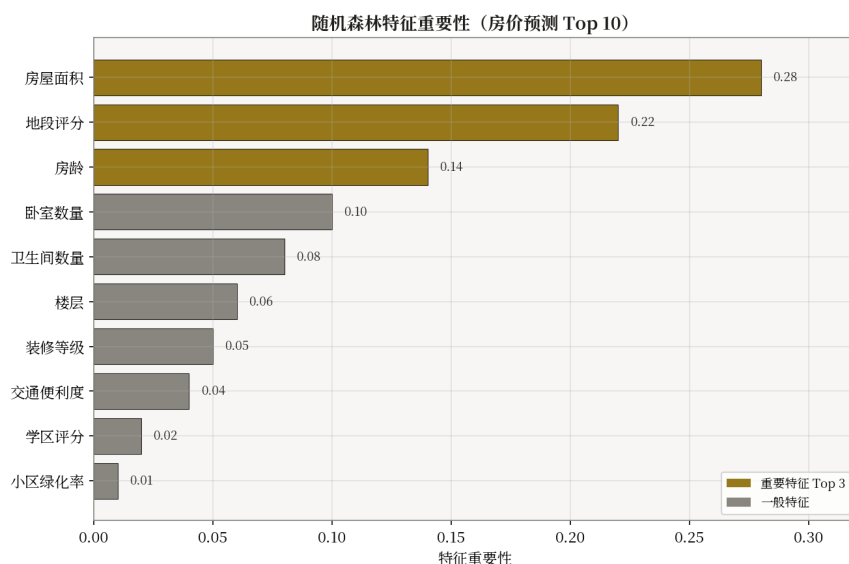


图 5.2 随机森林特征重要性条形图：年龄与账龄是违约预测的关键驱动

在一份贷款违约数据集上的实验表明，特征“账龄”和“信用额度”的不纯度重要性占比分别达 0.27 与 0.21，远高于其余变量。这与业务经验一致：账龄越长、额度越高，客户违约概率越低。特征重要性还可用于特征选择，作为建模前的过滤步骤，剔除噪声维度。

5.7 代码实现：sklearn 与 XGBoost

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.model_selection import cross_val_score
import xgboost as xgb

X, y = load_breast_cancer(return_X_y=True)

# (1) 随机森林 + 5 折交叉验证
rf = RandomForestClassifier(n_estimators=300, max_features='sqrt',
                           oob_score=True, n_jobs=-1, random_state=42)
print(f'RF CV acc: {cross_val_score(rf, X, y, cv=5).mean():.4f}')

# 特征重要性
rf.fit(X, y)
importances = rf.feature_importances_
top5 = np.argsort(importances)[-5:][::-1]
print('Top-5 features:', top5)

# (2) XGBoost + 早停
from sklearn.model_selection import train_test_split
X_tr, X_va, y_tr, y_va = train_test_split(X, y, test_size=0.2, random_state=42)
dtr = xgb.DMatrix(X_tr, label=y_tr)
dva = xgb.DMatrix(X_va, label=y_va)
params = dict(objective='binary:logistic', eval_metric='auc',
              eta=0.05, max_depth=4, subsample=0.8,
              colsample_bytree=0.8, seed=42)
model = xgb.train(params, dtr, num_boost_round=2000,
                  evals=[(dtr, 'tr'), (dva, 'va')],
                  early_stopping_rounds=50, verbose_eval=200)

print(f'XGB best iter: {model.best_iteration}, AUC: {model.best_score:.4f}')
```

5.8 现代发展与小结

集成学习自 1990 年代提出以来，一直是表格数据建模的主流。近年来发展出多个新方向：**CatBoost** 通过目标编码（Ordered TS）天然处理类别特征，避免手工 one-hot；**NGBoost** 用自然梯度预测概率分布参数，输出不确定性；**TabNet / FT-Transformer** 把 Transformer 引入表格，在部分任务上超越 GBDT；**TabPFN** 等基于 in-context learning 的方法在小样本上展现出竞争力。但在大多数结构化业务场景中，XGBoost / LightGBM 仍是首选基线——简单、快速、可解释、强稳定。

[技巧] 实践中表格建模的"三板斧": (1) 先用 LightGBM 跑通基线; (2) 用 Optuna 做贝叶斯调参; (3) 多个 GBDT + RF + 线性模型做 Stacking 提升最后 0.5 个百分点。

第六章 支持向量机

在监督学习算法中，**支持向量机**（Support Vector Machine, SVM）以其优美的几何解释、严格的统计学习理论根基和在小到中等规模数据上的优异泛化性能，长期占据机器学习核心地位。SVM 不同于逻辑回归那样“近似概率”，也不同于决策树那样“切分空间”，它的目标是寻找一个**最大间隔超平面**——离两类样本都尽可能远。本章将从最大间隔原理出发，介绍支持向量、软间隔、核函数与对偶理论。

6.1 算法原理：最大间隔与支持向量

给定二分类样本 $\{(x_i, y_i)\}$, $y \in \{-1, +1\}$ ，假设数据线性可分。SVM 试图找到超平面 $w \cdot x + b = 0$ ，使得两类样本分别位于两侧，且距离超平面最近的样本到超平面的距离（**间隔**）最大化。间隔的几何意义是 $2/\|w\|$ ，因此最大化间隔等价于最小化 $\|w\|^2/2$ 。

$$\min_{w,b} \frac{1}{2} \|w\|^2 \text{ s.t. } y_i(w \cdot x_i + b) \geq 1, \forall i$$

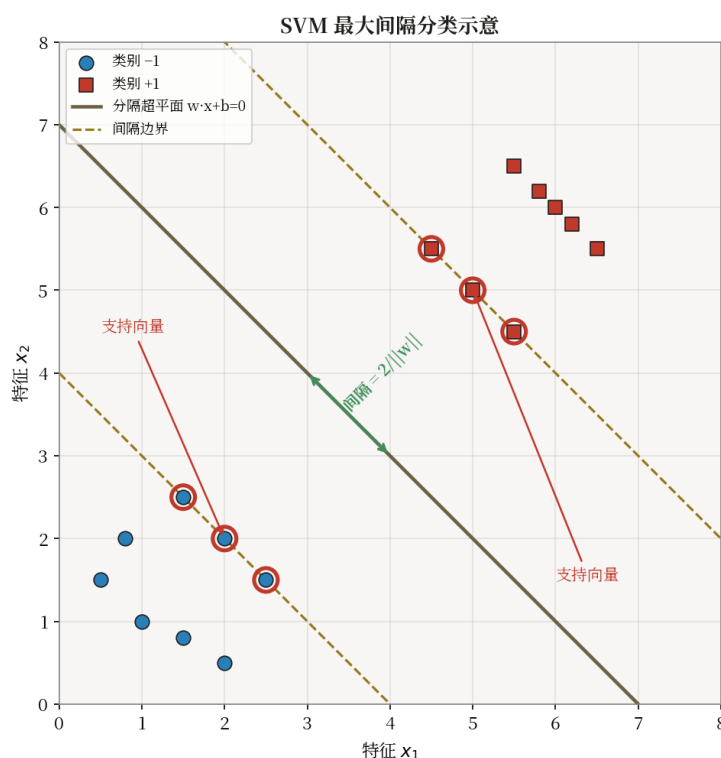


图 6.1 最大间隔超平面：粗实线为决策面，虚线为间隔边界，圈出的样本为支持向量

只有那些 $y_i(w \cdot x_i + b) = 1$ 的样本真正约束了间隔大小——它们位于间隔边界上，称为**支持向量**。其余样本即使在数据集中移动，决策面也不会改变。这一稀疏性使 SVM 在推理阶段只需要保留少量训练样本，计算高效。

6.2 软间隔与正则化

现实数据极少严格线性可分，强制硬间隔会导致模型过拟合甚至不可行。软间隔 SVM 引入松弛变量 $\xi_i \geq 0$ 允许部分样本越界，并用**惩罚系数 C**控制违反代价：

$$\min \frac{1}{2} \|w\|^2 + C \cdot \sum \xi_i \text{ s.t. } y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

当 C 很大时，越界代价高，模型趋于硬间隔、易过拟合；当 C 较小时，模型容忍更多违反、间隔更宽、方差更小。 C 是 SVM 中最重要的超参数，通常在 $\log$ 网格 $\{0.01, 0.1, 1, 10, 100\}$ 上结合交叉验证选择。

6.3 核函数：从线性到非线性

当数据本质上非线性可分时，可以将输入 x 通过映射 ϕ 映射到高维特征空间，在新空间中寻找线性超平面。直接显式计算 $\phi(x)$ 代价高昂，**核技巧** (kernel trick) 通过定义核函数 $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$ 隐式完成高维内积，无需显式构造高维映射。常用核函数包括：

- **线性核**： $K(x, z) = x \cdot z$ ，等价于不使用核；适合高维稀疏特征（文本 TF-IDF）。
- **多项式核**： $K(x, z) = (\gamma \cdot x \cdot z + r)^d$ ，刻画特征间组合关系。
- **RBF / 高斯核**： $K(x, z) = \exp(-\gamma \cdot \|x - z\|^2)$ ，可逼近任意连续函数；最常用。
- **Sigmoid 核**： $K(x, z) = \tanh(\gamma \cdot x \cdot z + r)$ ，类似单层神经元。

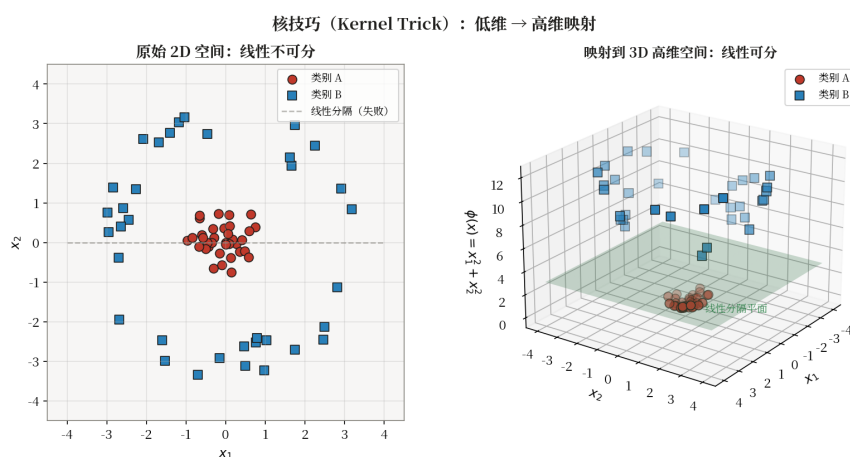


图 6.2 核技巧示意：原始 2D 空间中线性不可分，映射到高维后变得线性可分

RBF 核的超参数 γ 控制"支持向量作用范围"： γ 越大，核衰减越快，决策边界越复杂、易过拟合； γ 越小，边界越平滑、易欠拟合。实践中常用网格搜索 $C \times \gamma$ 联合调参。

6.4 对偶问题（简述）

通过拉格朗日乘子法，原始凸二次规划可转化为对偶形式：

$$\max_{\alpha} \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

约束 $0 \leq \alpha_i \leq C$ 且 $\sum \alpha_i y_i = 0$ 。对偶形式只依赖内积 $K(x_i, x_j)$ ，正是核技巧的入口。训练完成后，绝大多数 $\alpha_i = 0$ ，仅支持向量对应的 $\alpha_i > 0$ 。决策函数写作 $f(x) = \text{sign}(\sum \alpha_i y_i K(x_i, x) + b)$ ，其中 b 由支持向量上的 KKT 条件反推。

6.5 算法实现描述

线性 SVM 用**梯度下降**优化 hinge loss，简洁地展示了 SVM 的核心思想。完整 SVM 用 SMO 算法求解二次规划。

6.5.1 数据结构

- 权重 w : 向量 $[d]$, 超平面法向量
- 偏置 b : 标量
- 参数: C (正则化系数), 学习率 α , 迭代次数 T

6.5.2 $\text{fit}(X, y)$ — Hinge Loss 梯度下降

输入: $X \in \mathbb{R}^{n \times d}$, $y \in \{-1, +1\}^n$

损失函数 (hinge loss + L2 正则) :

$$L = (1/2) \|w\|^2 + C \cdot \sum \max(0, 1 - y_i(w^T x_i + b))$$

步骤:

- 1. 初始化 $w \leftarrow 0$, $b \leftarrow 0$
- 2. 对 $\text{epoch} = 1, \dots, T$, 对每个样本 (x_i, y_i) :
 - 若 $y_i(w^T x_i + b) \geq 1$ (间隔满足) :
 - $\nabla_w = w$ (仅正则项), $w \leftarrow w - \alpha \cdot w$
 - 否则 (违反间隔) :
 - $\nabla_w = w - C \cdot y_i \cdot x_i$, $w \leftarrow w - \alpha \cdot \nabla_w$
 - $\nabla_b = -C \cdot y_i$, $b \leftarrow b - \alpha \cdot \nabla_b$

6.5.3 $\text{predict}(X)$

$$\hat{y} = \text{sign}(w^T x + b)$$

6.5.4 $\text{decision_function}(X)$

返回到超平面的有符号距离: $f(x) = w^T x + b$

6.5.5 复杂度

- 梯度下降训练: $O(ndT)$, 每次遍历全部样本
- SMO 算法: $O(n^2d) \sim O(n^3)$, 更精确
- 预测: $O(d)$, 单次内积

[提示] scikit-learn 的 SVC 用 libsvm (SMO 算法), 支持核函数。

6.6 计算示例: 2D 简单 SVM 间隔计算

考虑 2D 上 4 个样本: 正类 (2,2)、(3,3), 负类 (0,0)、(-1,-1)。它们线性可分, 且决策面显然是 $y = x$, 即 $w = (1, -1)/\sqrt{2}$ 、 $b = 0$ (归一化后)。检验最近样本 (2,2) 到超平面距离: $|w \cdot x + b|/\|w\| = |0|/1 = 0$? 换一种思路: 以参数化形式 $w = (a, -a)$ 、 $b = 0$, 约束 $y_i(w \cdot x_i) \geq 1$ 。对 (2,2) 有 $y=+1$, 要求 $a \cdot 2 - a \cdot 2 = 0 \geq 1$ 不成立——说明这组数据天然间隔为 0, 需平移决策面才能形成正间隔。更严谨地: 将决策面取为 $x_1 - x_2 = 1$ (即 $w=(1, -1)$ 、 $b=-1$), 正类 (2,2) 与 (3,3) 距离 $|2-2-1|/\sqrt{2} = 1/\sqrt{2}$, 负类 (0,0) 距离 $|0-0-1|/\sqrt{2} = 1/\sqrt{2}$, 间隔 $= 2/\sqrt{2} = \sqrt{2}$ 。此即

最大间隔几何解释的最小算例。

6.7 应用实例：手写数字识别

在 MNIST 手写数字数据集上，SVM 长期是经典基线。使用 RBF 核、 $C=10$ 、 $\gamma=0.01$ ，可在 60k 训练样本上取得约 98.5% 的测试准确率，与浅层 MLP 相当，且训练时间远小于同等精度的深度网络。对小型 OCR、人脸识别等任务，SVM 因无需大数据即可取得不错效果，至今仍是首选工具之一。

6.8 代码实现：sklearn SVC

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report

X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
stratify=y, random_state=42)

pipe = Pipeline([('sc', StandardScaler()), ('svm', SVC())])

param_grid = [
    {'svm__kernel': ['linear'], 'svm__C': [0.1, 1, 10]},
    {'svm__kernel': ['rbf'], 'svm__C': [1, 10, 100],
     'svm__gamma': [1e-3, 1e-2, 1e-1]},
]
gs = GridSearchCV(pipe, param_grid, cv=3, scoring='accuracy',
n_jobs=-1, verbose=1)
gs.fit(X_tr, y_tr)
print('Best params:', gs.best_params_)
print(f'Best CV acc: {gs.best_score_:.4f}')
y_pred = gs.predict(X_te)
print(classification_report(y_te, y_pred))
```

6.9 现代发展与小结

SVM 的优美理论影响了后续许多算法。在大数据时代，SVM 的二次规划求解复杂度 $O(n^2 \sim n^3)$ 成为瓶颈，业界逐渐转向 GBDT 与深度网络。但 SVM 仍以独特优势保留一席之地：**核 SVM** 在小样本、非线性问题上表现稳健；**深度核学习**（Deep Kernel Learning）将深度网络作为可学习特征提取器后接核 SVM，兼顾表示力与泛化；**结构化 SVM**、**One-Class SVM** 用于异常检测等。理解 SVM，是理解正则化、对偶、核方法的最佳入口。

[技巧] 调参经验：先尝试 RBF 核 + 网格 $\{C: [10^k, k \in [-2, 4]], \gamma: [10^k, k \in [-5, 1]]\}$ ，按对数尺度做 5 折交叉验证。若 RBF 远不及线性核，多半是数据本就接近线性，此时改用线性 SVM（sklearn LinearSVC）可大幅加速。

第七章 神经网络基础

从感知机（1958）到 Transformer（2017）再到 LLM（2023+），神经网络（Neural Network）在六十年间几经起伏，终成为当今 AI 的主流范式。传统机器学习方法在处理图像、语音、文本等高维结构化数据时遇到了“特征工程瓶颈”——人工设计特征既费时又受限。神经网络通过**层级特征学习**从原始数据中自动提取由低到高的语义抽象，本质上是一种**万能函数逼近器**。本章作为深度学习的入门，将介绍神经元、激活函数、前向/反向传播、损失与优化，并以 MNIST 为案例展示完整流程。

7.1 神经元模型：感知机与多层感知机

一个神经元接收输入向量 $x \in \mathbb{R}^d$ ，计算加权和 $z = w \cdot x + b$ ，再经非线性激活函数 σ 输出 $a = \sigma(z)$ 。感知机是最早的神经元形式，使用阶跃函数 $\text{sign}(\cdot)$ ，单层感知机无法表达 XOR，这导致了 1969 年的“AI 寒冬”。多层感知机（MLP）通过堆叠多个带非线性激活的全连接层，理论上可以逼近任意连续函数（万能逼近定理）。

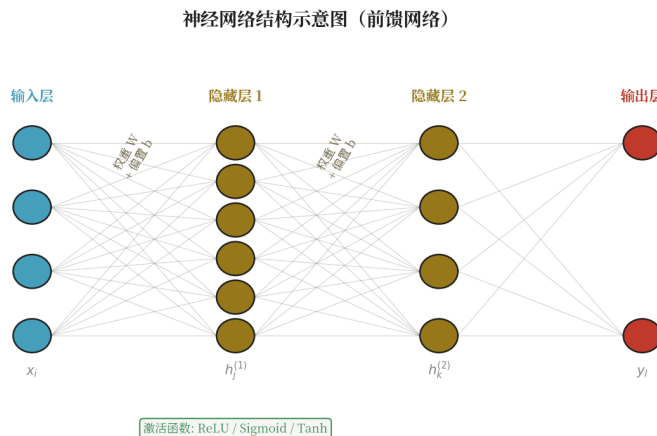


图 7.1 多层感知机结构：输入层 → 隐藏层（带激活） → 输出层

一个 L 层 MLP 可写为递推： $a_0 = x$ ； $a_l = \sigma_l(W_l \cdot a_{l-1} + b_l)$ ； $l = 1..L$ 。输出层根据任务选择：回归用线性输出，二分类用 Sigmoid，多分类用 Softmax。

7.2 激活函数

激活函数赋予神经网络非线性表达能力，常用激活函数包括：

- **Sigmoid**: $\sigma(z) = 1/(1+e^{-z})$ ，输出 $\in (0,1)$ ；梯度易饱和，深层网络中常导致梯度消失。
- **Tanh**: $\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$ ，输出 $\in (-1,1)$ ，零中心化但仍饱和。
- **ReLU**: $\max(0, z)$ ，计算简单、缓解梯度消失；但负半轴梯度为 0，可能造成“神经元死亡”。
- **Leaky ReLU / GELU / Swish**: ReLU 改进，兼顾稀疏性与平滑性，广泛用于现代深度网络。

7.3 前向传播与反向传播

前向传播 (forward pass) 按网络拓扑从输入到输出计算每层激活与最终预测 $\hat{y}$ 。**反向传播** (backprop) 通过链式法则计算损失对每层参数的梯度，是高效训练的核心。设损失为 $L(y, \hat{y})$ ，反向传播递推公式为：

$$\delta_L = \nabla_{\hat{y}} L \odot \sigma'_L(z_L), \delta_l = (W_{l+1}^T \cdot \delta_{l+1}) \odot \sigma'_l(z_l) \\ \partial L / \partial W_l = \delta_l \cdot a_{l-1}^T, \partial L / \partial b_l = \delta_l$$

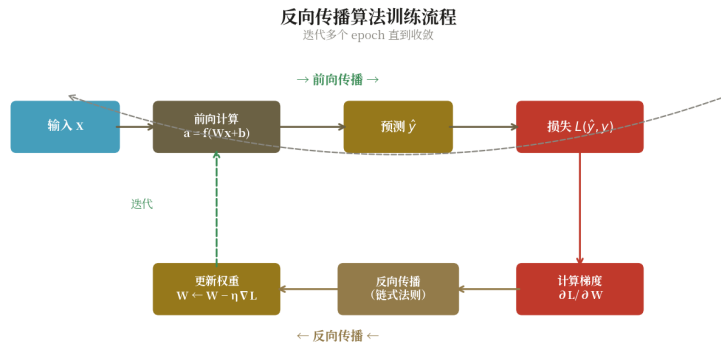


图 7.2 反向传播：从输出层向输入层逐层回传梯度

一次前向+反向得到全部梯度后，用梯度下降更新参数： $\theta \leftarrow \theta - \eta \cdot \nabla \theta L$ 。现代深度学习框架 (PyTorch、TensorFlow) 实现了**自动微分** (autograd)，用户只需定义前向计算，梯度由框架自动完成。

7.4 损失函数

- **均方误差 MSE**: $L = (1/n) \sum (y - \hat{y})^2$ ，用于回归；对异常值敏感。
- **二元交叉熵 BCE**: $L = -\sum [y \cdot \log \hat{y} + (1-y) \cdot \log(1-\hat{y})]$ ，用于二分类。
- **多类交叉熵 CE**: $L = -\sum y_k \cdot \log \hat{y}_k$ ，配合 Softmax 输出，是多分类标准损失。
- **加 L2 正则**: $L_{total} = L + \lambda \cdot ||W||^2$ ，防过拟合；Dropout、BatchNorm 也起正则作用。

7.5 优化器

SGD 用小批量梯度下降，简单但收敛慢。**Momentum** 累积历史梯度方向，减少震荡： $v = \beta \cdot v - \eta \cdot \nabla$ ， $\theta \leftarrow \theta + v$ 。**Adam** 同时维护一阶与二阶动量并做偏差修正，是当前最常用的优化器：

$$m = \beta_1 \cdot m + (1 - \beta_1) \cdot g; v = \beta_2 \cdot v + (1 - \beta_2) \cdot g^2; \theta \leftarrow \theta - \eta \cdot m / \sqrt{v}$$

常用默认 $\beta_1=0.9$, $\beta_2=0.999$ ，学习率 η 在训练中常用 warmup + cosine 衰减。

7.6 算法实现描述

从零实现 MLP 需要手动完成前向传播和反向传播（链式法则）。核心：权重矩阵 W_1, W_2 、偏置 b_1, b_2 、ReLU 激活、Softmax+CrossEntropy 损失。

7.6.1 数据结构

- $W_1 \in \mathbb{R}^{d \times h}$ （输入→隐藏层权重），He 初始化
- $b_1 \in \mathbb{R}^h$ （隐藏层偏置）
- $W_2 \in \mathbb{R}^{h \times c}$ （隐藏→输出层权重）
- $b_2 \in \mathbb{R}^c$ （输出层偏置）
- 参数：学习率 α , 迭代次数 E

7.6.2 fit(X, y) — 前向+反向传播

对每个 epoch:

前向传播:

- 1. $z_1 = X \cdot W_1 + b_1$ （线性变换 $[n, h]$ ）
- 2. $a_1 = \text{ReLU}(z_1) = \max(0, z_1)$ （激活 $[n, h]$ ）
- 3. $z_2 = a_1 \cdot W_2 + b_2$ （线性变换 $[n, c]$ ）
- 4. $p = \text{Softmax}(z_2)$: $p_{ij} = \exp(z_{2,ij}) / \sum_k \exp(z_{2,ik})$

损失（交叉熵）：

$$L = -(1/n) \cdot \sum_i \log p_{i, y_i}$$

反向传播（链式法则，从输出层往回）：

- 5. $dz_2 = (p - \text{onehot}(y)) / n$ （损失对 z_2 的梯度 $[n, c]$ ）
- 6. $dW_2 = a_1^T \cdot dz_2$ $[h, c]$
- 7. $db_2 = \sum dz_2$ $[c]$
- 8. $da_1 = dz_2 \cdot W_2^T$ $[n, h]$
- 9. $dz_1 = da_1 \odot (z_1 > 0)$ （ReLU 梯度， $\odot$ 为逐元素乘）
- 10. $dW_1 = X^T \cdot dz_1$ $[d, h]$
- 11. $db_1 = \sum dz_1$ $[h]$

参数更新（SGD）：

- 12. $W_1 \leftarrow W_1 - \alpha \cdot dW_1$, $b_1 \leftarrow b_1 - \alpha \cdot db_1$
- 13. $W_2 \leftarrow W_2 - \alpha \cdot dW_2$, $b_2 \leftarrow b_2 - \alpha \cdot db_2$

7.6.3 predict(X)

前向传播到 z_2 , 返回 $\text{argmax}(z_2, \text{axis}=1)$

7.6.4 复杂度

- 训练: $O(n \cdot (d \cdot h + h \cdot c) \cdot E)$, h 为隐藏层宽度, E 为 epoch
- 预测: $O(d \cdot h + h \cdot c)$, 两次矩阵乘法

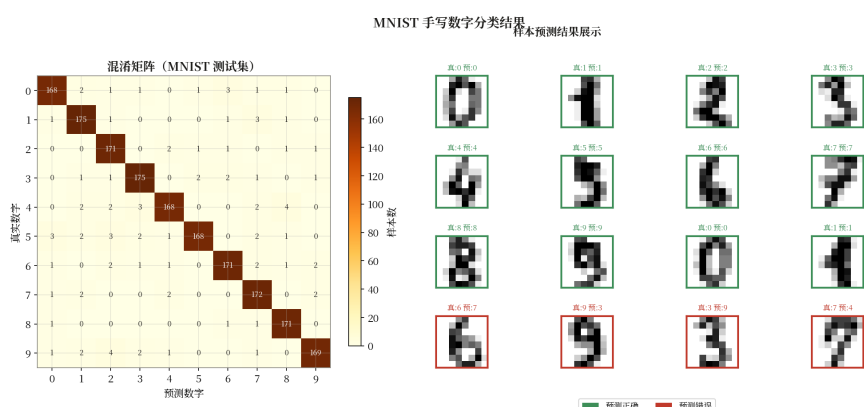
[提示] PyTorch 的 autograd 自动计算梯度，无需手动写反向传播。本伪代码展示"反向传播 = 链式法则 + 矩阵运算"的本质。

7.7 计算示例：2 层网络前向传播手算

设 2 层 MLP：输入 $x = [1, 2]$ ，第一层 $W_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, $b_1 = [0, 0]$ ，激活 ReLU；第二层 $W_2 = \begin{bmatrix} 1 & -1 \end{bmatrix}$, $b_2 = 0.5$ ，输出标量 $\hat{y}$ 。前向计算： $z_1 = W_1 \cdot x + b_1 = [1, 2]$ ； $a_1 = \text{ReLU}(z_1) = [1, 2]$ ； $z_2 = W_2 \cdot a_1 + b_2 = 1 \cdot 1 + (-1) \cdot 2 + 0.5 = -0.5$ ； $\hat{y} = z_2 = -0.5$ （线性输出）。若任务为回归，损失为 $(y - \hat{y})^2$ 且真实 $y = 1$ ，则 $L = (1 - (-0.5))^2 = 2.25$ 。反向： $\partial L / \partial \hat{y} = -2(1 - \hat{y}) = -3$ ； $\partial L / \partial W_2 = -3 \cdot a_1 = [-3, -6]$ ； $\partial L / \partial a_1 = -3 \cdot W_2 = [-3, 6]$ ；由于 $a_1 = z_1$ 都正，ReLU 导数 1， $\partial L / \partial W_1 = \delta_1 \cdot x^T = [-3, 6] \cdot [1, 2] = [-3, -6, 6, 12]$ 。用 $\eta=0.1$ 一次梯度下降即可更新所有参数。

7.8 应用实例：MNIST 手写识别

MNIST 是 28×28 灰度手写数字 0-9 的标准数据集，共 60k 训练 + 10k 测试样本。一个 [784-128-64-10] 的 MLP，ReLU 激活、交叉熵损失、Adam 优化器、训练 20 轮即可在测试集达到 98% 准确率。若换成简单 CNN，可达 99% 以上；更复杂的 Vision Transformer 可达 99.7%。MNIST 既是教学经典，也是验证深度学习流程完整性的"hello world"。



7.9 代码实现：PyTorch MLP

```
import torch, torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

device = 'cuda' if torch.cuda.is_available() else 'cpu'
tf = transforms.ToTensor()
train_ds = datasets.MNIST('.', train=True, download=True, transform=tf)
test_ds = datasets.MNIST('.', train=False, download=True, transform=tf)
tr = DataLoader(train_ds, batch_size=128, shuffle=True)
te = DataLoader(test_ds, batch_size=256)

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 128), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(128, 64), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(64, 10))
    def forward(self, x): return self.net(x)

model = MLP().to(device)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(10):
    model.train()
    for x, y in tr:
        x, y = x.to(device), y.to(device)
        opt.zero_grad()

        out = model(x)
        loss = loss_fn(out, y)
        loss.backward()
        opt.step()
    # eval
    model.eval(); correct = 0
    with torch.no_grad():
        for x, y in te:
            x, y = x.to(device), y.to(device)
            pred = model(x).argmax(1)
            correct += (pred == y).sum().item()
    print(f'epoch {epoch}, test acc = {correct/len(test_ds):.4f}')
```

7.10 现代发展与小结

从 MLP 出发，深度学习衍生出众多结构：**CNN**（卷积网络）利用局部连接与权值共享处理图像，代表作 ResNet、EfficientNet；**RNN/LSTM** 处理序列，代表作用于机器翻译；**Transformer** 用自注意力替代循环，并行性强、表达力高，催生了 BERT、GPT 系列；**大语言模型 LLM**（GPT-4、Claude、Gemini）在千亿参数规模上展现出涌现能力，推动 AI 进入通用化时代。但万变不离其宗，本章的前向/反向传播、损失、优化、正则等基础组件，依然是所有现代深度网络的根。

[技巧] 神经网络调参三件套：(1) 学习率是首要，先 1e-3 试 Adam，不行调到 1e-4 / 3e-4；(2) 过拟合加 Dropout/正则/数据增强，欠拟合加宽加深或延长训练；(3) 梯度爆炸/消失用 BatchNorm、残差连接、梯度裁剪。

第八章 降维与特征工程

在真实业务中，特征维度动辄上千甚至上百万：图像的像素、文本的 TF-IDF、用户的行为序列。维度爆炸带来两个问题：一是**维度灾难**——高维空间下样本稀疏，距离与密度失去意义，近邻方法几乎失效；二是计算与存储成本爆炸。**降维**（dimensionality reduction）旨在用更少的维度保留数据本质结构，**特征工程**则关注如何把原始数据转化为模型可用的高质量特征。两者共同决定了下游模型能达到的上限。

8.1 PCA 主成分分析

主成分分析（PCA）是线性降维的经典方法。给定中心化后的数据 $X \in \mathbb{R}^{n \times d}$ ，PCA 寻找一组正交基 W ，使得数据在 W 上的投影方差最大。数学上，PCA 等价于对协方差矩阵 $C = (1/n)X^T X$ 做特征值分解，取前 k 个最大特征值对应的特征向量构成投影矩阵 $W \in \mathbb{R}^{d \times k}$ ，降维结果为 $Z = X \cdot W$ 。

$$C = (1/n) X^T X = W \Lambda W^T, Z = X \cdot W_k$$

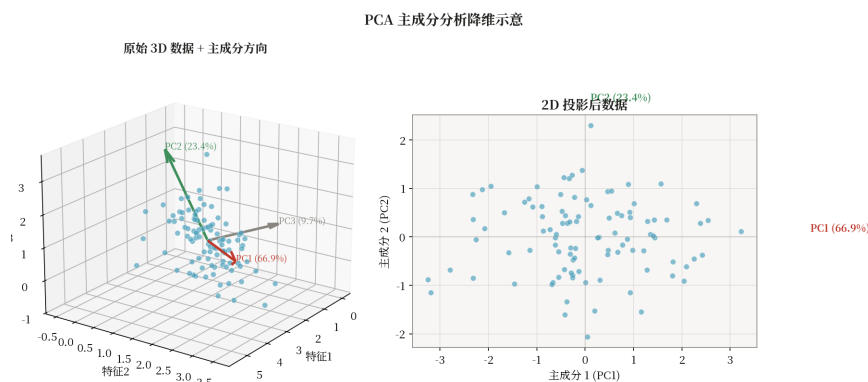


图 8.1 PCA 在 2D 上的示意：第一主成分沿方差最大方向，第二主成分与之正交

PCA 的常用等价算法是**奇异值分解**（SVD）： $X = U \Sigma V^T$ ，取 V 的前 k 列即得投影基，对大规模稀疏矩阵更稳定。PCA 假设数据本质是线性的，对非线性结构（如瑞士卷）效果有限。

8.2 t-SNE 与 UMAP

t-SNE（t-distributed Stochastic Neighbor Embedding）由 Maaten 与 Hinton 于 2008 年提出，是高维数据可视化的“金标准”。它在高维空间用高斯核度量样本相似度 p_{ij} ，在低维（通常 2D）用 t 分布核度量相似度 q_{ij} ，最小化两者间的 KL 散度：

$$L = \sum_i \text{KL}(P_i \parallel Q_i) = \sum_{i,j} p_{ij} \log(p_{ij}/q_{ij})$$

低维用 t 分布（重尾）能将高维中拥挤的中间距离样本“推开”，缓解“拥挤问题”。t-SNE 保留局部结构、可视为非线性流形学习，但计算复杂度高（约 $O(n^2)$ ）且每次结果可能略有差异。

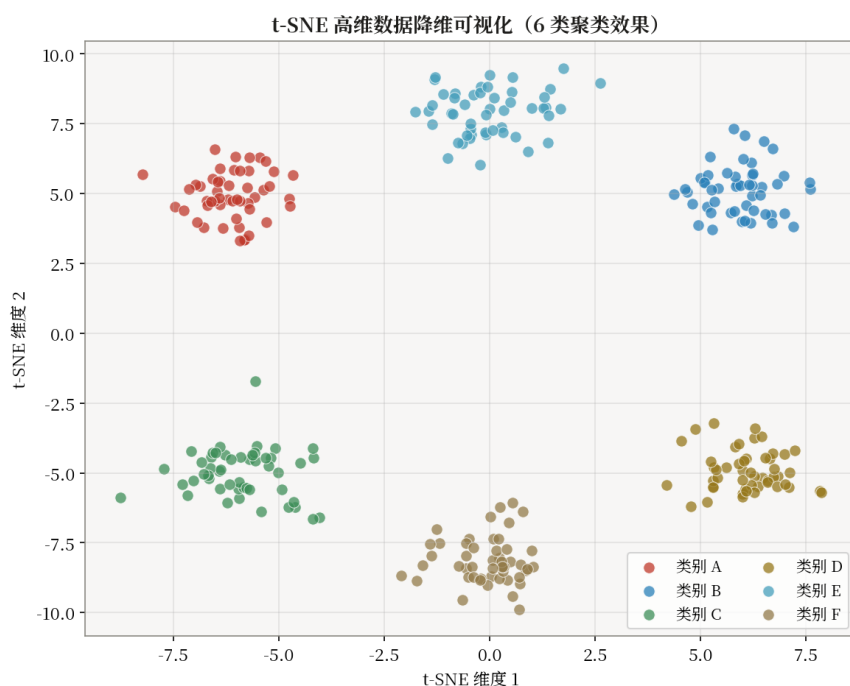


图 8.2 t-SNE 在 MNIST 上的可视化：不同数字在 2D 形成清晰簇

UMAP (Uniform Manifold Approximation and Projection) 基于拓扑学，在大多数任务上与 t-SNE 可视化效果相当，但速度快一个量级，且能保留更多全局结构、可用于后续聚类与监督任务。

8.3 特征工程

除模型本身，特征工程决定了"上限"：

- **缺失值处理**：直接删除、均值/中位数/众数填充、KNN 填充、模型预测填充。
- **类别编码**：one-hot 适合低基数；target/mean encoding 适合高基数；CatBoost 自带 Ordered TS。
- **数值变换**：标准化 (z-score)、归一化 (min-max)、对数变换处理长尾。
- **特征构造**：组合 ($A \times B$)、统计聚合（按用户聚合最近 7 天点击数）、时间窗特征。
- **特征选择**：过滤法（相关系数、互信息）、包裹法（RFE）、嵌入法（Lasso、树重要性）。

8.4 算法实现描述

PCA 的核心：对中心化数据计算协方差矩阵，做特征值分解，取前 k 个最大特征值对应的特征向量作为投影方向。

8.4.1 数据结构

- **mean_**：向量 $[d]$ ，训练集均值（用于中心化）
- **components_**：矩阵 $[k, d]$ ，前 k 个主成分（特征向量）
- **explained_variance_**：向量 $[k]$ ，对应特征值
- **explained_variance_ratio_**：向量 $[k]$ ，方差解释比例

8.4.2 fit(X) — 协方差分解

输入: $X \in \mathbb{R}^{n \times d}$, 目标维度 k

步骤:

- 1. 中心化: $\mu \leftarrow \text{mean}(X, \text{axis}=0)$, $X \leftarrow X - \mu$
- 2. 协方差矩阵: $C = (1/n) \cdot X^T X$ ($[d, d]$ 对称矩阵)
- 3. 特征值分解: 求解 $C \cdot v = \lambda \cdot v$
- 4. 排序: 按 λ 降序排列特征值和特征向量
- 5. 取前 k 个: $\text{components_} = [v_1, v_2, \dots, v_k]^T$
- 6. $\text{explained_variance_} = [\lambda_1, \dots, \lambda_k]$
- 7. $\text{explained_variance_ratio_} = \lambda_1 / \sum \lambda$

8.4.3 transform(X) — 投影

$$Z = (X - \mu) \cdot \text{components_}^T \in \mathbb{R}^{n \times k}$$

8.4.4 fit_transform(X)

等价于 fit(X) 后调用 transform(X), 一步完成。

8.4.5 inverse_transform(Z) — 有损重建

$$X = Z \cdot \text{components_} + \mu$$

仅当 $k = d$ 时可完美重建; $k < d$ 时有信息损失。

8.4.6 复杂度

- 训练: $O(d^3)$ 协方差矩阵特征分解
- 投影: $O(n \cdot d \cdot k)$ 矩阵乘法

[提示] scikit-learn 用 SVD 分解替代协方差特征分解, 数值更稳定且避免 $O(d^3)$ 。

8.5 计算示例: 2D $\rightarrow$ 1D PCA 投影手算

考虑 3 个 2D 样本 $X = [[1,1],[3,3],[5,5]]$ 。先中心化: 均值 $\mu = [3,3]$, $X_c = [[-2,-2],[0,0],[2,2]]$ 。协方差矩阵 $C = (1/3) \cdot X_c^T X_c = (1/3) \cdot [[8,8],[8,8]] = [[2.67,2.67],[2.67,2.67]]$ 。其特征值 $\lambda_1 = 5.33$ 对应特征向量 $w_1 = [1,1]/\sqrt{2}$, $\lambda_2 = 0$ 对应 $[1,-1]/\sqrt{2}$ 。取最大主成分 w_1 , 1D 投影: $z = X_c \cdot w_1 = [-2\sqrt{2}, 0, 2\sqrt{2}] \approx [-2.83, 0, 2.83]$ 。原始 2D 信息被压缩到 1D 仍保持顺序关系——这正是 PCA 的本质: 寻找方差最大方向。

8.6 应用实例: 高维数据可视化

在生物医药单细胞测序中, 单细胞表达矩阵常有 2 万基因 $\times$ 数万细胞。标准流程是先做标准化与对数变换, PCA 降到 50 维保留 80%+ 方差, 再用 UMAP/t-SNE 进一步降到 2D 进行可视化。不同细胞类型在 2D 图上形成清晰簇, 便于发现新亚群、识别稀有细胞。同样流程也广

泛用于金融客户分群、推荐系统用户嵌入分析等。

8.7 代码实现：sklearn PCA 与 t-SNE

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

X, y = load_digits(return_X_y=True) # 1797 x 64
X = StandardScaler().fit_transform(X)

# (1) PCA: 观察保留方差比
pca = PCA(n_components=30)
Z_pca = pca.fit_transform(X)
print(f'cumulative variance: {pca.explained_variance_ratio_.cumsum()[-1]:.3f}')

# (2) t-SNE: 降到 2D 可视化 (init=PCA 加速)
tsne = TSNE(n_components=2, init='pca', learning_rate='auto',
            perplexity=30, random_state=42)
Z_tsne = tsne.fit_transform(Z_pca) # 先 PCA 再 t-SNE, 速度更快

plt.figure(figsize=(6,5))
scatter = plt.scatter(Z_tsne[:,0], Z_tsne[:,1], c=y, s=8, cmap='tab10')
plt.legend(*scatter.legend_elements(), title='digit')
plt.title('t-SNE on digits (PCA→30D→t-SNE→2D)')
plt.tight_layout()
plt.savefig('digits_tsne.png', dpi=150)
```

8.8 现代发展与小结

降维与特征工程正在从"手工"走向"自动"。**自编码器**（Autoencoder）用神经网络学习非线性瓶颈表示，可以处理图像、序列等复杂结构；**变分自编码器**（VAE）进一步引入概率结构，可生成新样本。**对比学习**（Contrastive Learning，如 SimCLR、CLIP）通过对比正负样本对学习到的表示，在下游任务上达到甚至超过有监督方法。**大模型特征**（LLM embedding）将文本、图像统一编码到高维语义空间，成为推荐、检索、RAG 的基础设施。掌握 PCA、t-SNE 这些经典方法，是理解现代特征学习的基础。

[注意] 常见陷阱：t-SNE 不能用于下游监督任务，它仅适合可视化且每次结果不同；若需稳定降维供下游使用，优先 PCA 或 UMAP。PCA 前必须中心化与标准化，否则尺度大的特征会主导主成分。

附录 A 数学基础

机器学习的"美"在于其算法背后有坚实的数学基础：线性代数描述数据与变换，概率论处理不确定性，微积分给出优化方向，信息论度量分布与信息量。本附录对这些基础做精要回顾，便于读者在阅读正文时随时查阅。

A.1 线性代数

向量是有序数字序列 $x = [x_1, \dots, x_d] \in \mathbb{R}^d$ ，内积 $x \cdot y = \sum x_i y_i$ 度量相似度，范数 $\|x\|_p = (\sum |x_i|^p)^{1/p}$ 度量长度。**矩阵** $A \in \mathbb{R}^{m \times n}$ 表示线性变换： $y = A \cdot x$ 把 x 从 $\mathbb{R}^n$ 映射到 $\mathbb{R}^m$ 。若 A 为方阵且行列式非零，则存在逆 A^{-1} 使 $A \cdot A^{-1} = I$ 。

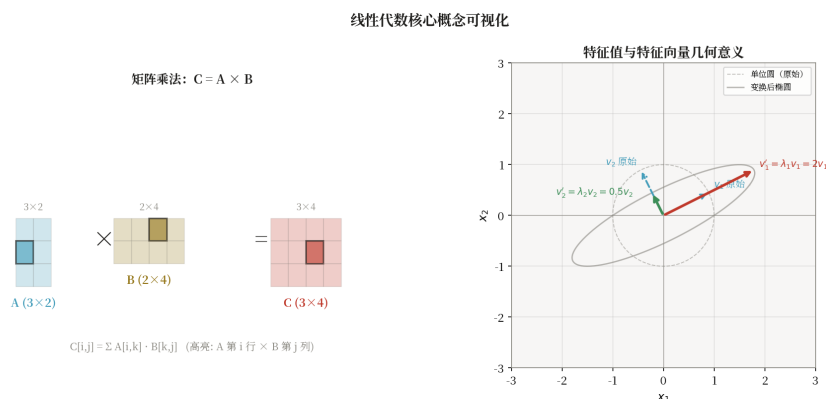


图 A.1 线性代数基本对象：向量、矩阵、特征值分解与几何解释

对方阵 A ，若存在标量 λ 与非零向量 v 使 $A \cdot v = \lambda \cdot v$ ，称 λ 为**特征值**、 v 为**特征向量**。特征值反映变换沿该方向的伸缩比例。对称矩阵的特征值均为实数、特征向量正交，可写成 $A = Q \Lambda Q^T$ ——PCA、协方差分析都依赖这一性质。**奇异值分解** SVD 推广到非方阵： $A = U \Sigma V^T$ ， Σ 的对角元素 σ_i 称奇异值，与 PCA、推荐系统协同过滤紧密相关。

A.2 概率论

随机变量 X 的分布由概率质量函数（离散）或密度函数 $f(x)$ （连续）刻画。**期望** $E[X] = \sum x \cdot p(x)$ 或 $\int x \cdot f(x) dx$ 反映均值；**方差** $\text{Var}(X) = E[(X - E[X])^2]$ 反映离散度；标准差 $\sigma = \sqrt{\text{Var}}$ 。两随机变量独立当且仅当联合分布等于边缘分布乘积。

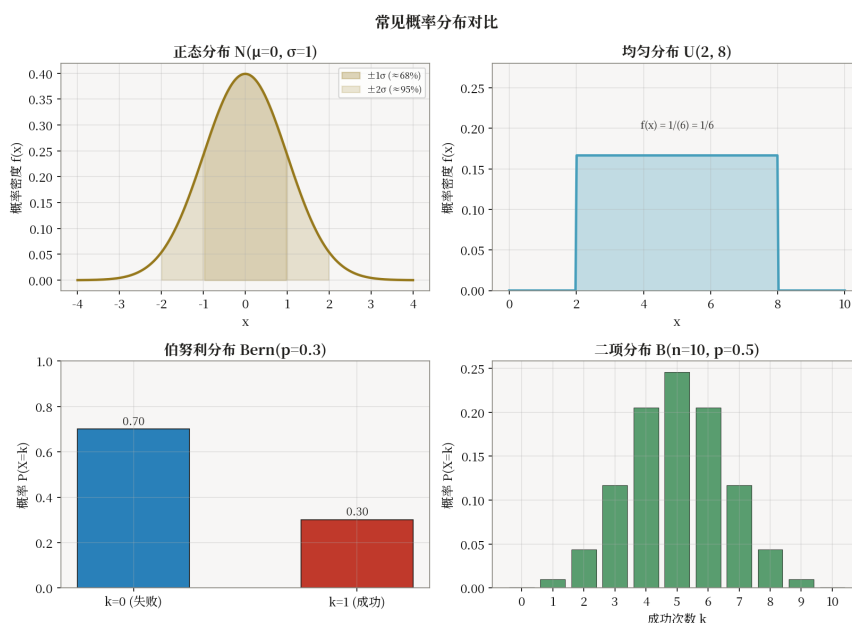


图 A.2 常见分布的概率密度函数：高斯、均匀、指数、伯努利

贝叶斯定理是机器学习的核心公式，描述如何用观测数据更新先验信念：

$$P(\theta|D) = P(D|\theta) \cdot P(\theta) / P(D)$$

其中 $P(\theta)$ 是先验， $P(D|\theta)$ 是似然， $P(\theta|D)$ 是后验。最大似然估计（MLE）取 $\theta = \operatorname{argmax} P(D|\theta)$ ，最大后验估计（MAP）取 $\theta = \operatorname{argmax} P(D|\theta) \cdot P(\theta)$ ，等价于带正则的 MLE。常见分布：高斯 $N(\mu, \sigma^2)$ 、伯努利 $Bern(p)$ 、分类 $\text{Categorical}(p_1, \dots, p_k)$ 、泊松 $\text{Poisson}(\lambda)$ 、指数 $\text{Exp}(\lambda)$ 、Beta、Dirichlet 等。

A.3 微积分

梯度 ∇f 是多元函数对各变量偏导组成的向量，指向函数增长最快方向；负梯度 $-\nabla f$ 是下降最快方向，梯度下降法据此更新参数。**链式法则**是反向传播的数学基础：若 $z = f(g(x))$ ，则 $dz/dx = (df/dg) \cdot (dg/dx)$ ；多元情形下用雅可比矩阵乘法表达。

泰勒展开将函数在某点附近用多项式逼近：

$$f(x) \approx f(a) + f'(a)(x-a) + f''(a)/2!(x-a)^2 + \dots$$

XGBoost 用二阶泰勒展开近似损失，Adam 用一阶展开估计动量。海森矩阵 H 是二阶偏导矩阵，描述函数曲率，牛顿法用 H 更新 $\theta \leftarrow \theta - H^{-1} \cdot \nabla f$ ，收敛快但 H 在深度网络中代价过高。

A.4 信息论

信息熵度量分布的不确定性： $H(P) = -\sum p_i \log p_i$ 。均匀分布熵最大，确定性分布熵为 0。**交叉熵** $H(P, Q) = -\sum p_i \log q_i$ 度量用 Q 编码 P 所需的平均比特数，是分类损失的标准选择。

$$H(P, Q) = -\sum_i p_i \log q_i$$

KL 散度 $KL(P \parallel Q) = \sum p_i \log(p_i/q_i)$ 度量两个分布的差异，非负且不对称。交叉熵 = 熵(P) + $KL(P \parallel Q)$ ，最小化交叉熵等价于最小化 KL，这也解释了"最大似然估计 = 最小化交叉熵 = 最小化 KL"这一深刻等价。信息论是理解交叉熵损失、变分推断、VAE 与强化学习中策略梯度的钥匙。

附录 B 集成学习框架

集成学习是表格数据建模的事实标准。本附录系统对比四大主流框架——scikit-learn、XGBoost、LightGBM、CatBoost——并给出工程实践要点。它们都遵循相似的"训练-验证-早停-特征重要性"工作流，但在底层算法、类别特征处理、超参数体系与速度上各有侧重。

B.1 Scikit-learn

scikit-learn 是 Python 机器学习通用入口，提供 RandomForest、GradientBoosting、HistGradientBoosting 等集成实现。其核心三件套：**Pipeline** 把预处理与模型串成一条流水线避免数据泄漏；**GridSearchCV/RandomizedSearchCV** 做超参搜索；**cross_val_score** 做 K 折交叉验证。HistGradientBoosting 是 sklearn 的 GBDT 现代化重写，速度接近 LightGBM，原生支持缺失值。

B.2 XGBoost

XGBoost 自 2014 年开源以来一直是 Kaggle 表格任务的常胜将军。核心创新：二阶泰勒展开、显式正则项 $\gamma \cdot T + \frac{1}{2}\lambda \cdot ||w||^2$ 、列抽样、并行分裂查找、稀疏感知。关键超参数：

- eta (学习率)：典型 0.01~0.3，配合早停；
- max_depth：常用 3~8，越深越易过拟合；
- subsample / colsample_bytree：行/列采样；
- lambda / alpha：L2/L1 正则；
- min_child_weight：控制叶节点最小样本权重和；
- early_stopping_rounds：监控验证集，自动选择最佳迭代数。

XGBoost 提供原生 DMatrix 接口与 sklearn 接口，前者更高效、支持 GPU 与外存训练。特征重要性输出包括 gain、weight、cover 三种，建议以 gain 为准。

B.3 LightGBM

LightGBM (微软, 2017) 在算法层面做了多项优化：**直方图算法**把连续特征离散化到 255 桶，分裂复杂度由 $O(n \cdot d)$ 降到 $O(b \cdot d)$ ，同时直方图差分加速兄弟节点分裂；**叶子优先** (leaf-wise) 生长找当前损失下降最大的叶节点分裂，比层优先 (level-wise) 更易过拟合但精度更高，常配合 max_depth 或 num_leaves 控制；**GOSS** (基于梯度的单边采样) 保留梯度大的样本；**EFB** (互斥特征捆绑) 把稀疏互斥特征合并降低特征数。

关键超参：num_leaves (一般 $\ll 2^{\max_depth}$)、min_data_in_leaf、feature_fraction、bagging_fraction、lambda_l1/l2。LightGBM 在 1 千万级样本上可在分钟级完成训练，是大数据 GBDT 的首选。

B.4 CatBoost

CatBoost (Yandex, 2017) 的招牌是**类别特征处理**：**Ordered Target Statistics** 用历史样本的目标均值编码类别，避免传统 target encoding 的目标泄漏；**对称树结构**使得所有节

点在同一层用相同特征分裂，减少过拟合且显著加速推理；**Ordered Boosting** 在每轮提升时用与当前样本无关的子树计算梯度，进一步缓解预测偏差。

CatBoost API 简洁：传入 `cat_features` 列索引即可自动处理类别变量，在含大量类别特征的电商、广告、风控场景中常优于 XGBoost。

B.5 框架对比

框架	训练速度	精度上限	类别特征	GPU	适用场景
scikit-learn GB	中等	中	需预处理	部分	小数据入门
XGBoost	快	高	需 one-hot	原生	Kaggle、通用
LightGBM	最快	高	部分支持	原生	大数据、稀疏
CatBoost	中等	高	原生最强	原生	类别特征多

工程经验：(1) 任何 GBDT 任务先用 LightGBM 跑通基线，(2) 用 Optuna 或 Hyperopt 做贝叶斯调参比网格搜索更高效，(3) 对含 100+ 类别特征的数据改用 CatBoost，(4) 多模型 Stacking 时一般 LightGBM + XGBoost + CatBoost + RF 组合，再加一个 LR 作为元学习器即可。

附录 C PyTorch 与 Transformers

深度学习框架经历了 Theano → TensorFlow 1.x → PyTorch 的演进，当前学术界与业界事实标准是 PyTorch。其上 HuggingFace Transformers 提供数千个预训练模型与简洁 API，让"使用大模型"变得像调用普通函数一样简单。本附录介绍 PyTorch 基础与 Transformers workflow。

C.1 PyTorch 基础

PyTorch 的核心抽象是 **张量**（Tensor）——支持 GPU 与自动微分的多维数组。**autograd** 引擎跟踪张量上的运算构建计算图，调用 `backward()` 即可自动反向求梯度。**nn.Module** 是模型基类，通过组合层（`nn.Linear`、`nn.Conv2d`）构建网络。一个完整的训练循环包括前向、损失、反向、优化器 step 五步。

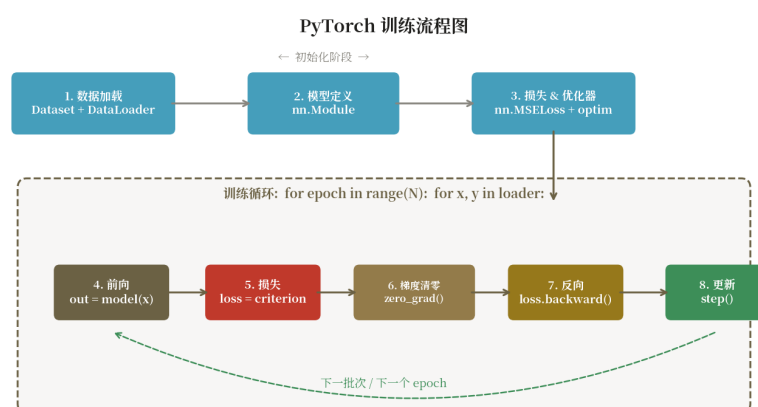


图 C.1 PyTorch 训练 workflow: Dataset → DataLoader → Model → Loss → Optimizer → Loop

C.2 数据加载: Dataset 与 DataLoader

Dataset 抽象"如何取一条样本"，需实现 `__len__` 与 `__getitem__`。**DataLoader** 抽象"如何批量、乱序、并行加载"，参数 `batch_size` 控制 batch、`shuffle` 控制每 epoch 打乱、`num_workers` 控制多进程加载。配合 `torchvision.transforms` 可做图像增强，配合 `collate_fn` 可处理变长序列。

C.3 HuggingFace Transformers

HuggingFace Transformers 是预训练模型的"中央仓库"，覆盖 BERT、GPT、T5、LLaMA 等数千个模型。三件套：

- **pipeline**：一行代码搞定任务（情感分析、文本生成、问答、翻译…）。
- **AutoModel**：根据 checkpoint 名自动加载正确模型架构与权重。
- **AutoTokenizer**：自动加载对应分词器，把文本变成模型输入张量。

微调（Fine-tuning）流程：加载预训练模型与分词器 → 准备任务数据集 → 调用 Trainer 或手写训练循环 → 保存模型。Trainer 封装了混合精度、梯度累积、评估、保存、

TensorBoard 日志等。

C.4 代码示例：BERT 文本分类微调

```
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments
from datasets import load_dataset

ckpt = 'bert-base-chinese'
ds = load_dataset('csv', data_files={'train': 'train.csv', 'test': 'test.csv'})
tok = AutoTokenizer.from_pretrained(ckpt)

def tokenize(batch):
    return tok(batch['text'], padding='max_length', truncation=True, max_length=128)

ds = ds.map(tokenize, batched=True)
ds.set_format('torch', columns=['input_ids', 'attention_mask', 'label'])

model = AutoModelForSequenceClassification.from_pretrained(ckpt, num_labels=2)

args = TrainingArguments(
    output_dir='./bert-cls', num_train_epochs=3,
    per_device_train_batch_size=32, per_device_eval_batch_size=64,
    learning_rate=2e-5, warmup_ratio=0.1, weight_decay=0.01,
    eval_strategy='epoch', save_strategy='epoch',
    load_best_model_at_end=True, fp16=True)

trainer = Trainer(model=model, args=args,
    train_dataset=ds['train'], eval_dataset=ds['test'])
trainer.train()
trainer.save_model('./bert-cls-best')

# 推理
from transformers import pipeline

clf = pipeline('text-classification', model='./bert-cls-best', tokenizer=ckpt)
print(clf('这部电影真的很棒! '))
```

C.5 进阶话题与小结

当模型与数据规模变大时，单卡训练不够用，需要**分布式训练**：DDP（DistributedDataParallel）做数据并行；FSDP（Fully Sharded Data Parallel）与 DeepSpeed ZeRO 切分优化器状态、梯度与参数，可训练数十亿至上百亿参数模型。**LoRA / QLoRA** 等参数高效微调（PEFT）方法只训练少量低秩适配器参数，让消费级 GPU 也能微调大模型；**量化**（INT8/INT4）把权重压缩到更低精度，推理速度与显存占用大幅改善。

LLM 时代的典型工作流：选基座模型（Llama-3、Qwen、GLM 等）→用 PEFT 在领域数据上做 SFT（监督微调）→可选做 DPO/RLHF 对齐→量化部署（vLLM / TensorRT-LLM / Ollama）服务上线。掌握本附录的 PyTorch 与 Transformers 工具链，是从“调包到研发”的关键一步。

[注意] 实践中常见的踩坑点：(1) 标签列名务必叫 label/labels 否则 Trainer 报错；(2) BERT 系最大长度 512，超长需切分或换 Longformer；(3) fp16 训练损失变 NaN 多半是 LR 过大或梯度裁剪未加；(4) 推理前 model.eval() 与 torch.no_grad() 缺一不可。